arXiv:2410.05433v3 [cs.RO] 16 Sep 2026

2Fast-2Lamaa: Large-Scale Lidar-Inertial Localization and Mapping with Continuous Distance Fields

Published in:
The International Journal of Robotics Research
DOI: 10.1177/02783649261451003
Available at: www.sagepub.com

Cedric Le Gentil[1], Raphael Falque[2], Daniil Lisus[1], and Timothy D. Barfoot[1]

Abstract

This paper introduces 2Fast-2Lamaa, a lidar-inertial state estimation framework for odometry, mapping, and localization. Its first key component is the optimization-based undistortion of lidar scans, which uses continuous IMU preintegration to model the system's pose at every lidar point timestamp. The continuous trajectory over 100-200ms is parameterized only by the initial scan conditions (linear velocity and gravity orientation) and IMU biases, yielding eleven state variables. These are estimated by minimizing point-to-line and point-to-plane distances between lidar-extracted features without relying on previous estimates, resulting in a prior-less motion-distortion correction strategy. Because the method performs local state estimation, it directly provides scan-to-scan odometry. To maintain geometric consistency over longer periods, undistorted scans are used for scan-to-map registration. The map representation employs Gaussian Processes to form a continuous distance field, enabling point-to-surface distance queries anywhere in space. Poses of the undistorted scans are refined by minimizing these distances through non-linear least-squares optimization. For odometry and mapping, the map is built incrementally in real time; for pure localization, existing maps are reused. The incremental map construction also includes mechanisms for removing dynamic objects. We benchmark 2Fast-2Lamaa on over 750km of public and self-collected datasets from both automotive and handheld systems. The framework achieves state-of-the-art performance across diverse and challenging scenarios, reaching odometry and localization errors as low as 0.22% and 0.06 m, respectively. The real-time implementation is publicly available at https://github.com/clegenti/2fast2lamaa.

Keywords

Localization, mapping, state estimation, lidar-inertial navigation

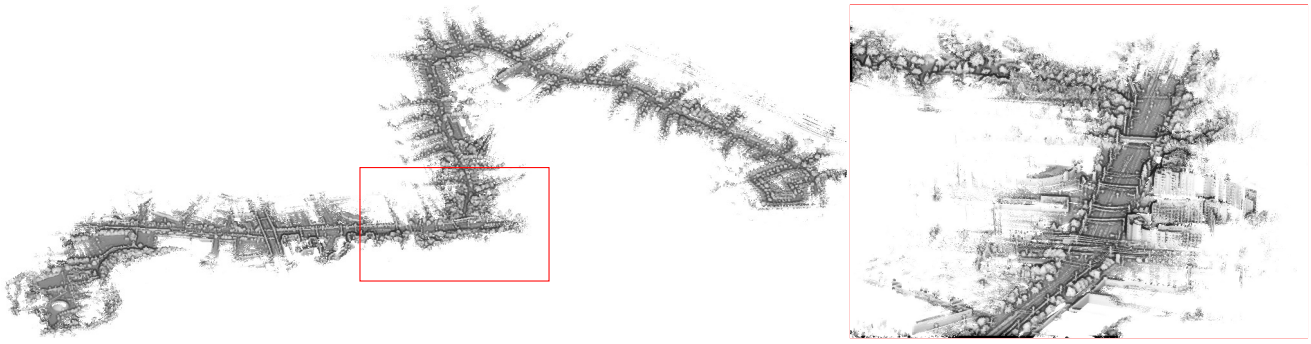
1 Introduction

Over the past decades, the robotics community has put a lot of effort into solving odometry, that is, estimating a system's ego-motion in unknown environments. In our previous work (Le Gentil et al. 2025), we showed that modern sensing capabilities provide highly accurate odometry even in challenging automotive scenarios. For the majority of real-world robot deployments, odometry alone is not sufficient to estimate the system's state. With the example of self-driving vehicles or autonomous conveying, the robot is expected to navigate between known locations in a previously mapped environment. In this context, the critical enabling component for autonomous operations is localization, not odometry. The latter is simply used as an initial guess/prior between localization steps. In this paper, we present **2Fast-2Lamaa**, which stands for **F**ast **F**ield-based **A**gent-**s**ubtracted **T**ruly coupled **L**idar **L**ocalization **a**nd **M**apping with **A**ccelerometer and **A**ngular-rate. This convoluted acronym refers to a lidar-inertial framework that addresses both mapping and localization by tightly coupling lidar and Inertial Measurement Unit (IMU) measurements and leveraging continuous distance fields to represent the environment.

The first step of most lidar-based systems is motion-distortion correction. Commonly used lidars do not capture instantaneous snapshots of the environment; instead, they sweep one or more laser beams through the scene to collect 3D scans. Accordingly, any motion of the sensor during a scan's duration creates motion distortion in the data. Many lidar-based state estimation algorithms leverage data from an IMU to address this issue (Lee et al. 2024). The most naive approach consists of using the latest pose and velocity estimates and integrating the inertial measurements to approximate the system's trajectory for the duration of the incoming scan. With that movement prediction, the scan can be undistorted before being used in a non-linear optimization for scan-to-scan or scan-to-map rigid registration, alongside inertial constraints between consecutive scans. While this approach has been qualified as 'tightly-coupled' lidar-inertial estimation, for example, by Ye et al. (2019), this undistortion strategy is a 'one-off' operation that can be thought of as an 'open-loop' process. Because it freezes the scans based on prior information, it decouples the problem of lidar data undistortion from the estimation of scan-to-scan

[1]Robotics Institute, University of Toronto Institute for Aerospace Studies, Canada
[2]Robotics Institute, University of Technology Sydney, Australia

Corresponding author:
Cedric Le Gentil, Robotics Institute University of Toronto Institute for Aerospace Studies, North York, Ontario M3H 5T6, Canada.
Email: cedric.legentil@robotics.utias.utoronto.ca



(a) 10M-point map of a $8\,\mathrm{km}$ trajectory (b) Zoom

Figure 1. 2Fast-2Lamaa performs odometry, mapping, and localization over large-scale environments. It relies on GP-based distance fields for scan-to-map registration. Thanks to efficient data structures, the map can contain details of the environment's geometry while allowing large-scale operations. The images here are visualizations of the map created with an $8\,\mathrm{km}$-long *Suburbs* sequence from the Boreas-RT dataset. Despite the length of the trajectory, the map can represent all the observed geometry (a), without sacrificing details (b).

motion. This can lead to unrecoverable errors if the prior state estimate is not accurate enough due to accumulated drift or outlier data. Other approaches to lidar-inertial state estimation consider the lidar points individually through some continuous motion representation. There, the IMU measurements can be used as residuals to constrain the continuous state (Talbot et al. 2025) or directly used to locally parameterize the trajectory (Le Gentil et al. 2018). These methods estimate the system's trajectory at the same time as correcting motion distortion in a *truly coupled* manner. Building upon our previous work (Le Gentil et al. 2024a), 2Fast-2Lamaa falls into the latter category by fully characterizing the trajectory during a scan based on IMU measurements without the need for any explicit motion model.

Once a scan is undistorted, 2Fast-2Lamaa performs localization by estimating a rigid transformation through scan-to-map registration. Note that when used for odometry or mapping, 2Fast-2Lamaa builds the map incrementally, whereas it leverages a previously built map when performing pure localization. A key component of a localization framework is the choice of map representation. The majority of lidar-based state estimation frameworks model the environment with point clouds (with or without normal vectors). In such a case, scan registration is generally performed with a variant of the Iterative Closest Point (ICP) algorithm (Besl and McKay 1992). 2Fast-2Lamaa differs from this paradigm as it leverages distance fields. These are continuous functions that can be queried at any location and that return an approximation of the Euclidean distance to the closest object/surface in the environment. Accordingly, the registration process consists of directly minimizing distance queries in a non-linear least-squares formulation, neither requiring explicit geometric primitives to model the environment nor any data association steps. The distance field presented in this work is based on Gaussian Process (GP) regression (Rasmussen and Williams 2006), similarly to Le Gentil et al. (2024b). While the concept of a GP-based distance field is not new, 2Fast-2Lamaa is the first real-time state estimation framework that successfully builds upon this idea for large-scale operations (sequences over $10\,\mathrm{km}$-long), as illustrated in Fig. 1. Note that this integration is not trivial, as standard GP regression suffers from cubic computational complexity. By leveraging efficient data structures and computing GPs locally, 2Fast-2Lamaa's novel distance field approximation displays a $\log(N)$ computational complexity. The efficient data structures also enable real-time dynamic object removal through simple ray tracing.

Regardless of the choice of geometric representation (point cloud, mesh, distance field, etc.), mapping and localization frameworks can opt for different high-level mapping strategies. The most natural option is to use a single globally consistent map. However, it has been demonstrated that accurate localization does not require geometrically consistent global maps (Brooks 1987; Baumgartner and Skaar 1994). Using a topometric approach that only requires local geometric consistency, Furgale and Barfoot (2010) performs navigation by moving in a set of topologically connected submaps, alleviating the need for perfect odometry/trajectory estimation when building the map. This approach is especially suited for *Teach and Repeat* tasks, where the robot is manually piloted to map the environment (teach/mapping) before performing autonomous path tracking to follow the original trajectory (repeat/localization). Our proposed framework can operate with either a globally consistent map or in a topometric manner, utilizing a succession of submaps to localize the robot's repeating trajectories.

As any odometry pipeline is bound to drift, building maps purely on odometry estimates lead to inconsistencies at the global scale when considering trajectories that include loops that revisit previously explored areas. While not the core focus of the present work, 2Fast-2Lamaa can also be used for globally consistent trajectory estimation by performing loop-closure detection and batch pose-graph optimization, similarly to Simultaneous Localisation And Mapping (SLAM) frameworks. This process is performed online when running odometry with submaps (topometric mapping mode). The proposed loop-closure detection and correction pipeline relies on the projection of each submap into 2D image-like data structures that represent the environment's elevation changes. Similar to the work by Giubilato et al. (2022), visual features are extracted

from these image-like structures, matched to detect loop-closures, and then used to provide the associated rough SE(2) geometric transformation between submaps. After SE(3) submap-to-submap pose refinement with the aforementioned GP-based distance fields, the global pose of each submap is estimated in a batch pose-graph optimization.

To summarise, our contributions are:

- The integration of truly coupled lidar-inertial motion-distortion correction in a localization and mapping framework named 2Fast-2Lamaa.
- The development of an efficient GP-based distance field for large-scale online mapping in the presence of dynamic objects.
- The integration of an online loop-closure detection and correction mechanism to extend 2Fast-2Lamaa's capabilities beyond odometry and incremental mapping.
- An extensive odometry and localization evaluation of 2Fast-2Lamaa with more than $750\,\mathrm{km}$ of automotive and handheld data.
- An open-source real-time implementation.

2 Related work

2.1 Lidar state estimation

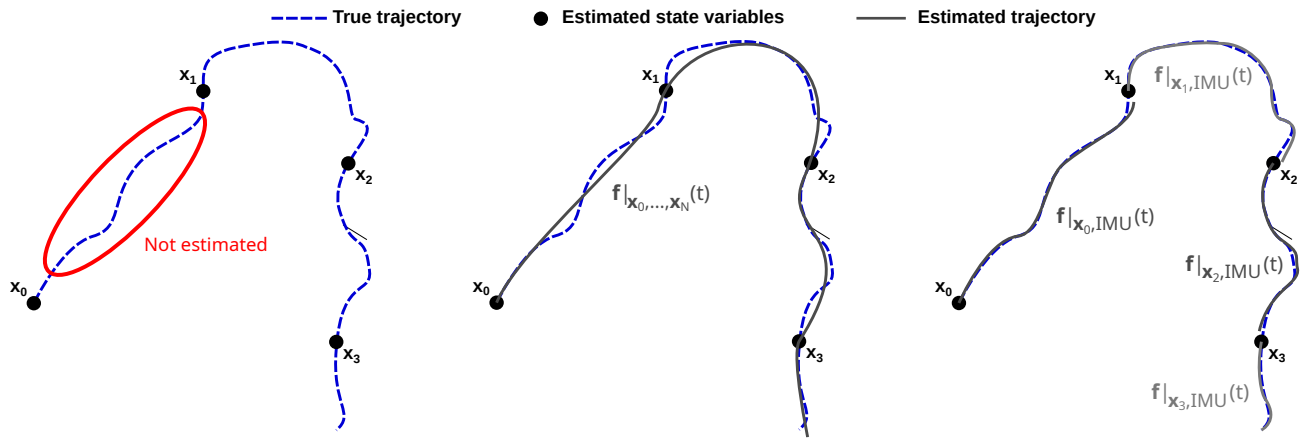
There are many different approaches to state estimation in the robotics literature. In this section, we mainly consider optimization-based lidar(-inertial) frameworks. The majority of such methods can be classified into three distinct state estimation categories illustrated in Fig. 2: (a) discrete, (b) continuous, and (c) hybrid. As mentioned in the introduction, motion distortion is addressed differently depending on the state representation. The first one (a) dissociates the act of undistortion from state estimation. Thus, motion correction is an open-loop process that preprocesses the scans based on previous estimates, and state estimation is discretely solved at the starting timestamp of each scan. With (b) and (c), state estimation and motion distortion are coupled in a single problem that estimates the continuous trajectory of the sensor (at least locally). Fully continuous approaches (b) generally assume a certain motion model to represent the sensor's movement with a continuous function. IMU measurements can be used as residuals or control inputs of the system's trajectory. Hybrid approaches (c) attempt to leverage the best of both (a) and (b) by locally parameterizing the trajectory based on inertial information, while keeping discrete variables at the global scale for simplicity and computational efficiency. The rest of this subsection provides a brief literature review of the three types of methods

Considered as state-of-the-art odometry frameworks, Fast-LIO2 (Xu et al. 2022) and DLIO (Chen et al. 2023) are examples of one-off undistortion processes based on discrete-state estimation. The former uses an iterated Kalman filter, with the core registration step consisting of optimizing point-to-plane residuals between the open-loop-undistorted scans and a global map built incrementally. DLIO introduces a more precise way to undistort the point clouds using a constant-jerk motion model for continuous integration of the IMU measurements. After motion-distortion correction, DLIO performs a variant of Generalized-ICP (Segal et al. 2009) and uses a "hierarchical geometric observer" (Lopez 2023) to estimate the system's pose efficiently. Note that DLIO and Fast-LIO2 only optimize for the last sensor pose. Other works leverage a similar undistortion process but optimize over a window of poses in a factor-graph formulation (Ye et al. 2019; Shan et al. 2020). Note that the aforementioned frameworks require sufficient geometric features in the system's surroundings. To address geometrically degenerated environments, COIN-LIO (Pfreundschuh et al. 2024) proposes to leverage intensity information from the lidar data to extract distinctive features even on flat surfaces. More discrete-state lidar-inertial pipelines are discussed in a recent survey paper (Lee et al. 2024).

As shown in another recent survey paper (Talbot et al. 2025), there is a wide variety of continuous-time state formulations for robotics. Some of the early lidar(-inertial) work built upon continuous-time representations that make strong assumptions about the motion. For example, Zebedee (Bosse et al. 2012) uses piece-wise linear functions to model the system's trajectory. This corresponds to a strict assumption of constant velocity between control points. Using inertial residuals, it demonstrated the ability to map a 3D environment using a randomly oscillating 2D lidar. Later, LOAM (Zhang and Singh 2014) used the same constant velocity assumption with both actuated 2D and 3D lidars. This assumption is still used in recent frameworks like CT-LIO (Dellenbach et al. 2022) or MOLA (Blanco-Claraco 2025). A key contribution of LOAM was the introduction of planar and edge features extracted from the raw lidar data for efficient registration. This concept has been reused in many works, including LeGo-LOAM (Shan and Englot 2018) and the present work (using a different extraction method).

Throughout the years, other continuous representations have been used to model more complex lidar trajectories. A recurrent formulation is based on splines (Droeschel and Behnke 2018; Cao et al. 2025). The benefit of splines with respect to discrete state variables has been demonstrated in the context of vision-based SLAM by Cioffi et al. (2022) at the cost of a higher computational cost. Another branch of continuous-time estimation is based on a particular class of sparse GPs (Barfoot 2024). This approach elegantly models the trajectory based on a probabilistic motion model. For both lidars and radars, continuous GP states with a white-noise-on-acceleration motion prior have demonstrated high levels of accuracy and real-time computation (Burnett et al. 2025a). Note that different motion priors are possible, as demonstrated by Tang et al. (2019) with a white-noise-on-jerk prior. And inertial data can also be used as control inputs (Burnett et al. 2025b; Lilge and Barfoot 2025).

The hybrid approach shown in Fig. 2(c) relies on IMU measurements to locally represent the system's trajectory. An issue with inertial-based motion prediction/characterization is the strong dependence on initial conditions and biases when integrating and double-integrating the gyroscope and accelerometer measurements. To prevent the reintegration of the IMU data every time the initial conditions are updated during the state optimization process, Lupton and Sukkarieh (2012) introduced the concept of preintegration. It corresponds to the creation of pseudo measurements that combine the information from multiple IMU measurements without the need to know the initial conditions. Numerous



(a) Discrete-time estimation (b) Continuous-time estimation (c) Hybrid estimation

Figure 2. Most optimization-based lidar state estimation frameworks can be classified into three categories. (a) represents discrete-time estimation where the system's pose is estimated at a finite set of timestamps. The trajectory between timestamps is not estimated. (b) leverages a function over the whole duration of operations to represent the motion continuously. Such a method generally relies on some motion model and is parameterized by a set of supporting points. (c) shows a hybrid approach that locally characterizes the trajectory continuously based on inertial data, but uses discrete state variables at the global scale. This approach does not require an explicit motion model, but the global trajectory shows discontinuities when switching to a new timestamp. 2Fast-2Lamaa is built on the latter paradigm.

works have improved on the original preintegration method (Forster et al. 2017; Eckenhoff et al. 2019; Yang et al. 2020). The inertial chapter of the SLAM Handbook provides an overview and comparison of several of these works (Huang et al. 2026). Note that, unlike the original preintegration use case, the hybrid state estimation approach does not attempt to combine IMU measurements into a small number of pseudo-measurements but rather requires an 'upsampling' of the inertial information for it to be available at any timestamp. This approach was originally introduced to address the issue of lidar-IMU extrinsic calibration by first upsampling the raw inertial readings before performing preintegration at a higher frequency (Le Gentil et al. 2018). Later approaches elegantly addressed preintegration under the scope of continuous state representation (Le Gentil et al. 2020b; Le Gentil and Vidal-Calleja 2023), enabling full-batch lidar-inertial localization and mapping (Le Gentil et al. 2021). As this hybrid approach is well-suited to asynchronous inertial-aided estimation, other works with event cameras (Le Gentil et al. 2020a; Li et al. 2024) and radars (Hatleskog et al. 2025) have adopted preintegration as their state representation. 2Fast-2Lamaa also uses continuous preintegration to characterize the system's motion during each lidar scan. The corresponding discrete state is the gravity vector orientation, the linear velocity at the beginning of the scan, and the IMU biases, resulting solely in 11-Degree-of-Freedom (DoF) to optimize.

2.2 Map representation

Traditionally, robotic maps consist of geometric landmarks jointly estimated with the system's pose (Dissanayake et al. 2001). Over the past decades, we have seen a constant increase in sensor bandwidth and computational power. Naturally, the robotics community has harnessed these new hardware capabilities with algorithms that build and use denser and denser maps of the environment. Nowadays, the most common representation for lidar state estimation consists of voxel maps where each cell stores the centroid and normal vector of all the points that occurred in the cell. Fast-LIO2 (Xu et al. 2022) is an example of such methods, leveraging point-to-plane registration constraints. Other works, such as KISS-ICP (Vizzo et al. 2023) and KISS-SLAM (Guadagnino et al. 2025), chose to use point-to-point residuals, but suffer from significantly lesser performance on benchmarks such as the Newer College dataset (Ramezani et al. 2020). Other frameworks keep more information in each cell, for example, the local distribution of the points with normal distributions (Biber and Strasser 2003; Magnusson 2009). Another way to model the environment is to use sets of surface primitives. Surfels might be the simplest of these primitives and have been used in numerous lidar frameworks (Bosse et al. 2012; Droeschel and Behnke 2018; Park et al. 2018). Some recent works deal with more complex primitives by directly building and using a mesh of the environment (Lin et al. 2023; Ruan et al. 2023).

The aforementioned representations (point clouds, surfels, meshes, etc.) store information about the environment's surface. While memory-efficient, they contain limited information about the rest of the space explored by the system. Volumetric approaches, on the other hand, store information over the whole space and can track occupancy, Truncated Signed Distance Field (TSDF), etc. These are less explored for lidar-based estimation, but are crucial for autonomous navigation (e.g., to plan safe trajectories through the environment). Voxblox (Oleynikova et al. 2017) and FIESTA (Han et al. 2019) are prime examples of such mapping techniques. Leveraging advances in computer graphics, many mapping frameworks have been built atop the OpenVDB structure (Museth et al. 2013; Museth 2013). Some fuse information from multiple scans at the TSDF level (Vizzo et al. 2022), where others use the occupancy (Zhu et al. 2021), or the Euclidean distance field (Wu et al. 2025). Focusing on state estimation, VoxGraph (Reijgwart et al. 2020) extends VoxBlox (Oleynikova et al. 2017) and achieves real-time performance. It leverages SDF submaps

to compute submap-to-submap pose constraints, which are then integrated into a pose-graph optimization framework. More specific to lidar systems, D-LIO (Coto-Elena et al. 2026) uses a fast truncated distance field stored in a multi-level hashmap to perform scan-to-map registration. While the authors claim scalability to large-scale environments, their experiments on the VBR dataset (Brizi et al. 2024) show a large RAM requirement that cannot be fulfilled without removing information from memory. Another interesting work is shown by Boche et al. (2025) with OKVIS2-X, where vision-based estimation can be complemented with lidar-to-occupancy-map factors (Boche et al. 2024) using the Supereight2 data structure (Funk et al. 2021). Focusing on exploration, Schmid et al. (2021) also leverage volumetric mapping with a submap-based strategy to account for large state estimate drift.

With the democratization of GPU computing, many robotics works have explored the use of neural representations and Gaussian splatting for map representation. Point-SLAM (Sandström et al. 2023) and Gs-icp SLAM (Ha et al. 2024) are examples of both approaches for SLAM with RGB-D sensors. DeepSDF (Park et al. 2019) proposes an auto-decoder architecture to learn and infer the signed distance function at object-scale. Later, iSDF (Ortiz et al. 2022) demonstrates the ability to model and learn such a field online for room-scale environments. While less popular, neural representations can be used for lidar-based state estimation. LocNDF (Wiesmann et al. 2023) addresses the localization problem with a neural field close to a TSDF, but cannot be run at sensor framerate, even in small-scale environments, while Nerf-LOAM (Deng et al. 2023) performs odometry and mapping in large-scale environments. More recently, PIN-SLAM (Pan et al. 2024) showcases top performance on the VBR dataset (Brizi et al. 2024) via TSDF modelling with real-time operations. PINGS (Pan et al. 2025) builds upon PIN-SLAM by coupling Gaussian splats with neural distance fields for improved rendering capacities using both lidar and vision-based sensing.

2Fast-2Lamaa follows a line of work that mixes both surface-based and volumetric information. Similar to the former, it only stores information on the surface of elements in the environment, but allows for the query of an Euclidean distance field approximation over the whole space without large memory requirements. This trend originated from the goal of surface reconstruction with GP implicit surfaces (Williams and Fitzgibbon 2006), which models a signed distance field close to the objects' surface. Later, Wu et al. (2021) shows the ability to approximate the distance to the closest surface anywhere in space by applying a non-linear operation over a GP-inferred field. It is demonstrated with offline experiments that such GP-based approaches enable lidar state estimation and planning (Wu et al. 2023). Le Gentil et al. (2024b) significantly improves the distance approximation while alleviating the original tradeoff between accuracy and surface interpolation. It represents the foundation of the fast distance field derived in the present work, which does not consider a single and inefficient GP, but breaks down the distance field modelling into many small local GPs. Consequently, our new field formulation enables registration in kilometre-long maps in real-time. In contrast, the original work by Le Gentil et al. (2024b) can barely run object-level odometry at 2 Hz.

2.3 Dynamic object rejection

A common assumption of many robotic state estimation algorithms is to consider the system's environment to be static. This assumption is very rarely verified in real-world applications. While robust techniques like RANSAC or m-estimators enable a robot to deal with a certain level of dynamicity in the scene (considering dynamic object points as outliers), there has been a growing interest in performing dynamic object detection to create maps that only contain static elements (Yoon et al. 2019; Schmid et al. 2023; Falque et al. 2023; Jia et al. 2024a; Duberg et al. 2024; Wu et al. 2024). However, most methods that focus on state estimation in dynamic environments first classify the points before performing standard scan registration (Pfreundschuh et al. 2021), and optionally tracking objects (Jia et al. 2024b). Other methods simply detect dynamic objects after lidar data registration (Lichtenfeld et al. 2024; Le Gentil et al. 2024a), relying on robust estimators to 'ignore' dynamic objects during the scan-alignment process. An exception to these two-step approaches is BTSA (Chen et al. 2025), which introduces a dynamic-aware ICP algorithm that couples the problems of dynamic point detection and state estimation in a single process. Another method, HiMo (Zhang et al. 2025), does not address the problem of ego-motion distortion, but tackles the moving object point cloud undistortion through scene flow estimation. We refer the reader to the relevant chapter of the SLAM Handbook (Schmid et al. 2026) for a deeper dive into state estimation in dynamic and deformable environments.

As the problem of dynamic object detection is not the core focus of 2Fast-2Lamaa, the proposed optimization-based motion-distortion correction step relies on robust loss functions to consider dynamic objects as outlier information. However, after scan-to-map registration, dynamic points can be removed from the map in an online or offline manner through some sort of ray tracing, similarly to the work from Pomerleau et al. (2014). In our ablation study, we evaluate the impact of the presence of dynamic points in the map on localization.

3 Framework overview

Let us consider a 6-DoF IMU (3-axis gyroscope and 3-axis accelerometer) and a 3D lidar rigidly mounted together. The lidar collects 3D points denoted as $\mathbf{x}_L^j$. The gyroscope measures the IMU angular velocity ω, and the accelerometer the proper acceleration $\mathbf{f}$. The homogeneous transformation $\mathbf{T}_I^L$ represents the extrinsic calibration between the two sensors. We aim to estimate the pose of the IMU $\mathbf{T}_W^{I_t}$ at time t with respect to a world frame $\mathfrak{F}_W$, with or without a prior map of the environment. The timestamp of the start of a lidar scan is denoted τ_i, and $\mathfrak{F}_{I_t}$ refers to the IMU frame at time t. As shown in Fig. 3, the proposed framework consists of two main components: one module for tightly-coupled lidar-inertial motion-distortion correction and a second one for scan-to-map localization.

To undistort lidar scans, 2Fast-2Lamaa first extracts lidar features from the raw scans and performs continuous

preintegration with the IMU data. Using continuous IMU preintegration, the system's trajectory is parameterized by the gravity vector $\mathbf{g}_{\tau_i}$ in $\mathfrak{F}_{I_{\tau_i}}$, the velocity $\mathbf{v}^{\tau_i}_{I_{\tau_i}}$ at time τ_i in $\mathfrak{F}_{I_{\tau_i}}$, and both the accelerometer and gyroscope biases $\mathbf{b}^i_f$ and $\mathbf{b}^i_\omega$ (considered constant for the duration of the scan). After associating features together, the motion during the lidar scans is estimated by minimizing lidar feature distances (point-to-plane and point-to-line, depending on the feature's nature) in a non-linear least-squares formulation. The output of this module consists of motion-corrected lidar point clouds $\mathcal{U}_i$ and associated scan-to-scan incremental motion $\tilde{\mathbf{T}}^{I_{\tau_{i+1}}}_{I_{\tau_i}}$.

For trajectory estimation, the localization module integrates the aforementioned incremental motion estimates and refines the global pose $\tilde{\mathbf{T}}^{I_{\tau_i}}_{W}$ of $\mathcal{U}_i$ with scan-to-map registration. The proposed framework can build the map $\mathcal{M}$ incrementally (mapping mode) or directly leverage an existing map of the environment (localization mode). Using GP regression, the map provides an approximation of the scene's Euclidean distance field. With the ability to efficiently query the distance $d(\mathbf{x})$ to the closest surface for any $\mathbf{x} \in \mathbb{R}^3$, the registration is the direct minimization of d for a subset of points from $\mathcal{U}_i$. Once an undistorted scan is registered to the map, it can be used to update the underlying data structure: extending the area covered by $\mathcal{M}$ or removing elements that are no longer present in the environment (such as dynamic objects or structural changes). Note that for repeating trajectories, both localization and mapping can be performed in a topometric manner by considering a chain of connected submaps instead of a unique global map. When mapping the environment with submaps, an optional online loop-closure detection and correction mechanism, presented in Section 6.3, can be used for globally consistent trajectory estimation.

4 Undistortion-based odometry

The undistortion module in 2Fast-2Lamaa is based on previous work for map-less and initialization-free dynamic object detection (Le Gentil et al. 2024a). The main differences are a novel approach for feature extraction and a change in the definition of the temporal window used for motion estimation. 2Fast-2Lamaa offers a faster front-end and reduces the time delay in the state computation by estimating the motion for the last incoming lidar scan, not waiting for subsequent scans as originally done.

4.1 IMU preintegrated continuous state

To undistort the incoming lidar data, let us consider the lidar points and IMU readings that have been collected over a short period of time. For convenience and implementation efficiency, this temporal window is chosen to span over two lidar scans (each defined as a full revolution for spinning sensors), with the previous scan from τ_{i-1} to τ_i and the current scan from τ_i to τ_{i+1}. As formulated for SE(3) by Forster et al. (2017), the rotation, velocity and translation preintegrated measurements from time τ_{i-1}, $\Delta\mathbf{R}^t_{\tau_i}$ and $\Delta\mathbf{p}^t_{\tau_i}$, correspond to

$$
\begin{aligned}
\Delta\mathbf{R}^t_{\tau_{i-1}} &= \prod_{\tau_{i-1}}^{t} \left(\mathrm{Exp}(\omega(\mathrm{t}) - \mathbf{b}_\omega(\mathrm{t}))\right)^{d\mathrm{t}}, \\
\Delta\mathbf{v}^t_{\tau_{i-1}} &= \int_{\tau_{i-1}}^{t} \Delta\mathbf{R}^s_{\tau_{i-1}} (\mathbf{f}(\mathrm{t}) - \mathbf{b}_f(\mathrm{t})) ds \\
\Delta\mathbf{p}^t_{\tau_{i-1}} &= \int_{\tau_{i-1}}^{t} \int_{\tau_{i-1}}^{\mathrm{t}} \Delta\mathbf{R}^s_{\tau_{i-1}} (\mathbf{f}(\mathrm{t}) - \mathbf{b}_f(\mathrm{t})) ds d\mathrm{t},
\end{aligned}
\tag{1}
$$

with $\mathbf{b}_\omega$ and $\mathbf{b}_f$ slow-varying additive biases. Using the Linear Preintegrated Measurements (LPMs) (Le Gentil and Vidal-Calleja 2023) to compute (1) based on piece-wise linear continuous representation of the inertial data, the IMU pose and velocity through time are defined as

$$
\begin{aligned}
\mathbf{T}^{I_t}_{I_{\tau_{i-1}}} &= \begin{bmatrix} \mathbf{R}^t_{I_{\tau_{i-1}}} & \mathbf{p}^t_{I_{\tau_{i-1}}} \\ \mathbf{0} & 1 \end{bmatrix} \quad \text{with} \quad \mathbf{R}^t_{I_{\tau_{i-1}}} = \Delta\mathbf{R}^t_{\tau_{i-1}}, \\
\mathbf{v}^{I_t}_{I_{\tau_{i-1}}} &= \mathbf{v}^{I_{\tau_{i-1}}}_{I_{\tau_{i-1}}} + (t - \tau_{i-1})\mathbf{g}_{\tau_{i-1}} + \Delta\mathbf{v}^t_{\tau_{i-1}}. \\
\mathbf{p}^{I_t}_{I_{\tau_{i-1}}} &= (t - \tau_{i-1})\mathbf{v}^{I_{\tau_{i-1}}}_{I_{\tau_{i-1}}} + \frac{(t - \tau_{i-1})^2}{2}\mathbf{g}_{\tau_{i-1}} + \Delta\mathbf{p}^t_{\tau_{i-1}}.
\end{aligned}
\tag{2}
$$

Note that the preintegrated measurements $\Delta\mathbf{R}^t_{\tau_{i-1}}$ and $\Delta\mathbf{p}^t_{\tau_{i-1}}$ are computed using a prior knowledge of the biases $\bar{\mathbf{b}}^{i-1}_f$ and $\bar{\mathbf{b}}^{i-1}_\omega$. Similar to the original on-manifold preintegration work (Forster et al. 2017), the preintegrated measurements are corrected through a first-order Taylor expansion to account for the unknown nature of the biases:

$$
\begin{aligned}
\Delta\mathbf{R}^t_{\tau_{i-1}} &\approx \bar{\Delta\mathbf{R}}^t_{\tau_{i-1}} \mathrm{Exp}\left(\frac{\partial\Delta\mathbf{R}^t_{\tau_{i-1}}}{\partial\mathbf{b}^{i-1}_\omega}\Delta\mathbf{b}^{i-1}_\omega\right), \\
\Delta\mathbf{v}^t_{\tau_{i-1}} &\approx \bar{\Delta\mathbf{v}}^t_{\tau_{i-1}} + \frac{\partial\Delta\mathbf{v}^t_{\tau_{i-1}}}{\partial\mathbf{b}^{i-1}_\omega}\Delta\mathbf{b}^{i-1}_\omega + \frac{\partial\Delta\mathbf{v}^t_{\tau_{i-1}}}{\partial\mathbf{b}^{i-1}_f}\Delta\mathbf{b}^{i-1}_f, \\
\Delta\mathbf{p}^t_{\tau_{i-1}} &\approx \bar{\Delta\mathbf{p}}^t_{\tau_{i-1}} + \frac{\partial\Delta\mathbf{p}^t_{\tau_{i-1}}}{\partial\mathbf{b}^{i-1}_\omega}\Delta\mathbf{b}^{i-1}_\omega + \frac{\partial\Delta\mathbf{p}^t_{\tau_{i-1}}}{\partial\mathbf{b}^{i-1}_f}\Delta\mathbf{b}^{i-1}_f,
\end{aligned}
\tag{3}
$$

with $\Delta\mathbf{b}^{i-1}_f = \mathbf{b}^{i-1}_f - \bar{\mathbf{b}}^{i-1}_f$, and $\Delta\mathbf{b}^{i-1}_\omega = \mathbf{b}^{i-1}_\omega - \bar{\mathbf{b}}^{i-1}_\omega$. Thus, the system's trajectory during the temporal window is fully characterized by $\mathcal{S} = \{\mathbf{g}_{\tau_{i-1}}, \mathbf{v}^{I_{\tau_{i-1}}}_{I_{\tau_{i-1}}}, \mathbf{b}^{i-1}_f, \mathbf{b}^{i-1}_\omega\}$. Accordingly, a lidar point $\mathbf{x}_L$ collected at time t can be projected to the $\mathfrak{F}_{I_{\tau_{i-1}}}$ frame as

$$
\begin{bmatrix} \mathbf{x}_{I_{\tau_{i-1}}} \\ 1 \end{bmatrix} = \mathbf{T}^{I_\mathrm{t}}_{I_{\tau_{i-1}}} \mathbf{T}^L_I \begin{bmatrix} \mathbf{x}_L \\ 1 \end{bmatrix}. \tag{4}
$$

4.2 Lidar features and data association

Using the raw lidar point clouds, features are extracted independently in the two scans that constitute the temporal window $[\tau_{i-1}, \tau_{i+1}]$. Similarly to the work by Zhang and Singh (2014), features can be *planar* or *edge* points. However, the feature point selection does not follow any type of curvature/roughness score. The planar points $\mathcal{P}_{i-1}$ and $\mathcal{P}_i$ are obtained by voxel-subsampling the corresponding scans and retaining only one point per voxel (random selection among the points in each voxel). For the edge points, the proposed method considers each lidar channel/ring independently and detects jumps in range values between consecutively collected points. The latter form the sets of edges $\mathcal{E}_{i-1}$ and $\mathcal{E}_i$. Note that the sets $\mathcal{P}_i$ and $\mathcal{E}_i$ are mutually

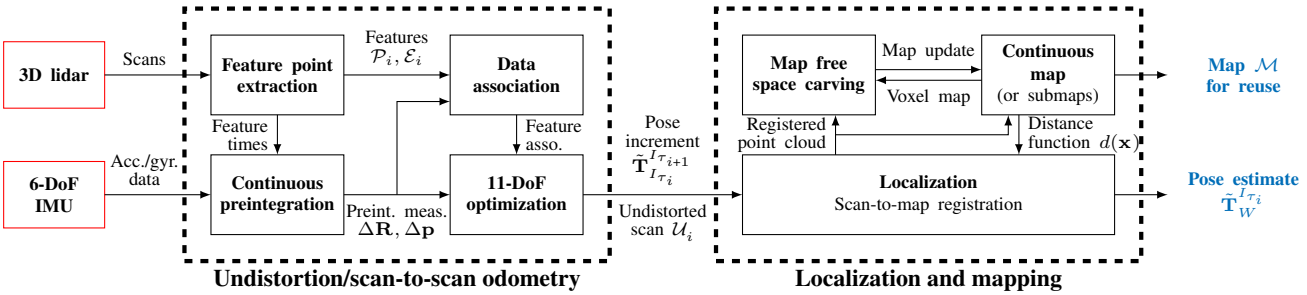


Figure 3. 2Fast-2Lamaa consists of two functional blocks. The first one is an optimization-based motion correction module that undistorts lidar scans using continuous IMU preintegration to characterize the system motion with only 11-DoFs. Once corrected, the scans are used for scan-to-map registration to estimate the global pose of the system.

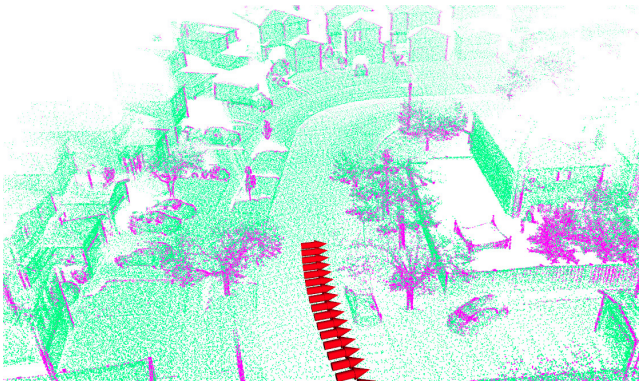


Figure 4. Accumulated lidar features (planar in green, edge in magenta) over 3 s of data in a *Suburbs* sequence from the Boreas-RT dataset.

exclusive. Fig. 4 shows an example of features obtained over 3 s of data in a suburban environment.

Then, the features from the two scans are associated together, between $\mathcal{P}_{i\text{-}1}$ and $\mathcal{P}_i$ or between $\mathcal{E}_{i\text{-}1}$ and $\mathcal{E}_i$, with a k-closest-neighbour search that enforces some conditions that depend on the feature type. For planar points, each point in $\mathcal{P}_i$ is associated with three features from $\mathcal{P}_{i\text{-}1}$, enforcing a minimum spread in terms of distance and angle, to avoid colinearity (a minimum of 0.05 m and 75° in our implementation). Similarly, each edge feature in $\mathcal{E}_i$ is matched with two sufficiently spaced points from $\mathcal{E}_{i\text{-}1}$ (typically 0.05 m). For each point, the association begins with a radius search that provides the set of points $\mathcal{A}$. Starting with the closest point in $\mathcal{A}$, the process iteratively tests the type-specific conditions for each point in $\mathcal{A}$ by order of distance, until the required number of valid points is reached (successful association) or when there are no more points to test in $\mathcal{A}$ (failed association).

4.3 Motion correction

The motion is estimated via a non-linear least-squares optimization of point-to-plane and point-to-line distances

$$\tilde{\mathcal{S}} = \underset{\mathcal{S}}{\operatorname{argmin}} \sum_{\mathbf{x}_L^j \in \mathcal{P}_i} {r_{\mathcal{P}_j}}^2 + \sum_{\mathbf{x}_L^j \in \mathcal{E}_i} {r_{\mathcal{E}_j}}^2, \tag{5}$$

where $r_{\mathcal{P}_j}$ is the point-to-plane distance between a planar feature $\mathbf{x}_L^j$ and the associated trio $\mathbf{x}_L^k$, $\mathbf{x}_L^l$, and $\mathbf{x}_L^m$, and $r_{\mathcal{E}_j}$ is the point-to-line distance between an edge feature $\mathbf{x}_L^j$ and the associated pair $\mathbf{x}_L^k$ and $\mathbf{x}_L^l$. Using (2) and (4) for point projection, the residuals are computed as

$$r_{\mathcal{P}_j} = \frac{(\mathbf{x}_{I_{\tau_{i\text{-}1}}}^j - \mathbf{x}_{I_{\tau_{i\text{-}1}}}^k)^\top \left((\mathbf{x}_{I_{\tau_{i\text{-}1}}}^k - \mathbf{x}_{I_{\tau_{i\text{-}1}}}^l) \times (\mathbf{x}_{I_{\tau_{i\text{-}1}}}^k - \mathbf{x}_{I_{\tau_{i\text{-}1}}}^m)\right)}{\|(\mathbf{x}_{I_{\tau_{i\text{-}1}}}^k - \mathbf{x}_{I_{\tau_{i\text{-}1}}}^l) \times (\mathbf{x}_{I_{\tau_{i\text{-}1}}}^k - \mathbf{x}_{I_{\tau_{i\text{-}1}}}^m)\|}, \tag{6}$$

and

$$r_{\mathcal{E}_j} = \frac{\|(\mathbf{x}_{I_{\tau_{i\text{-}1}}}^j - \mathbf{x}_{I_{\tau_{i\text{-}1}}}^k) \times (\mathbf{x}_{I_{\tau_{i\text{-}1}}}^j - \mathbf{x}_{I_{\tau_{i\text{-}1}}}^l)\|}{\|\mathbf{x}_{I_{\tau_{i\text{-}1}}}^k - \mathbf{x}_{I_{\tau_{i\text{-}1}}}^l\|}. \tag{7}$$

As the magnitude of the gravity vector $\mathbf{g}_{\tau_{i\text{-}1}}$ is known, the optimization (5) leverages the 2-sphere manifold, resulting in a total of eleven DoFs. It is solved using the Levenberg-Marquardt algorithm. After convergence, the estimated state $\tilde{\mathcal{S}}$ and (2) allow the computation of $\tilde{\mathbf{T}}_{I_{\tau_i}}^{\mathrm{t}}$ to correct the current scan's motion distortion and provide the scan-to-scan pose increment $\tilde{\mathbf{T}}_{I_{\tau_i}}^{I_{\tau_{i+1}}}$. The undistorted point cloud $\mathcal{U}_i$ and $\tilde{\mathbf{T}}_{I_{\tau_i}}^{I_{\tau_{i+1}}}$ are passed to the localization module (next section) for global pose estimation. The velocity and gravity estimates projected to τ_i, as well as an average of the past bias estimates, are used as the state's initial guess for the next temporal window.

The undistortion process presented in this section is performed for every incoming scan and estimates the continuous trajectory between $\tau_{i\text{-}1}$ and τ_{i+1}. It is done without fixing the motion between $\tau_{i\text{-}1}$ and τ_i in any way. Computing the motion for the duration of two scans at every scan creates an overlap in the consecutive temporal windows. That is why, at each step, the module solely outputs the transformation between τ_i and τ_{i+1}, ignoring the estimate between $\tau_{i\text{-}1}$ and τ_i. It is important to note that the trajectory over a small temporal window (typically 200 ms) is often close to translation-only movement, or can be close to rotation-only in some scenarios. These motions are not informative enough to make the accelerometer observable, as there is an ambiguity with gravity, as detailed by Tereshkov (2015). To address this issue, a weak zero-mean prior residual $\mathbf{r}_{\mathbf{b}_f} = c\mathbf{b}_f^{i\text{-}1}$, with c a constant, is added to prevent unrealistically high accelerometer bias estimates. Thus, (5) becomes

$$\tilde{\mathcal{S}} = \underset{\mathcal{S}}{\operatorname{argmin}} \sum_{\mathbf{x}_L^j \in \mathcal{P}_i} {r_{\mathcal{P}_j}}^2 + \sum_{\mathbf{x}_L^j \in \mathcal{E}_i} {r_{\mathcal{E}_j}}^2 + \|\mathbf{r}_{\mathbf{b}_f}\|^2. \tag{8}$$

5 Localization

5.1 Scan-to-map registration

Given a motion-corrected point cloud $\mathcal{U}_i$, 2Fast-2Lamaa estimates the scan-to-map transformation $\mathbf{T}_W^{I_{\tau_i}}$ using a map

$\mathcal{M}$ (given or incrementally built as detailed in Section 6), which provides a distance field $d(\mathbf{x})$ that can be queried for any point $\mathbf{x}$. As the undistortion step provides decent pose increment estimates, we do not estimate $\mathbf{T}_W^{I_{\tau_i}}$ directly, but perform a right-hand perturbation

$$\mathbf{T}_W^{I_{\tau_i}} = \tilde{\mathbf{T}}_W^{I_{\tau_{i-1}}} \tilde{\mathbf{T}}_{I_{\tau_{i-1}}}^{I_{\tau_i}} \begin{bmatrix} \mathrm{Exp}(\Delta\mathbf{r}) & \Delta\mathbf{p} \\ \mathbf{0} & 1 \end{bmatrix}, \tag{9}$$

with $\tilde{\mathbf{T}}_W^{I_{\tau_{i-1}}}$ the global pose estimate of the previous scan, $\tilde{\mathbf{T}}_{I_{\tau_{i-1}}}^{I_{\tau_i}}$ the pose increment from the undistortion step, and $\Delta\mathbf{r}$ and $\Delta\mathbf{p}$ the rotational and translational perturbation, respectively. Thus, using (9), the global registration estimates

$$\tilde{\Delta\mathbf{r}}, \tilde{\Delta\mathbf{p}} = \underset{\Delta\mathbf{r},\Delta\mathbf{p}}{\mathrm{argmin}} \sum_{\mathbf{x}_j \in \mathcal{U}_i} \left(d\left(\mathbf{T}_W^{I_{\tau_i}} \begin{bmatrix} \mathbf{x}_j \\ 1 \end{bmatrix} \right) \right)^2 \tag{10}$$

using the Levenberg-Marquardt algorithm. The key of 2Fast-2Lamaa is the computation of the distance field d. It is detailed in Section 6.

5.2 Gravity-alignment residual

Inspired by Noh et al. (2025), the scan-to-map registration (10) can be complemented with a gravity-alignment residual $\mathbf{r}_{g_i}$, when used for odometry or mapping, as opposed to pure localization. First the gravity vector $\mathbf{g}_W$ in the original sensor frame at τ_0 is estimated alongside the system's linear velocities $\mathbf{v}_W^{I_{\tau_i}}$ and IMU biases $\mathbf{b}_f$ and $\mathbf{b}_\omega$, using the first M estimated poses $\mathbf{T}_W^{I_{\tau_i}}$, obtained through scan-to-map registration. M is programmatically chosen to guarantee that enough translation and rotation occurred in the trajectory to make the accelerometer biases observable. Defining the optimized state $\mathcal{S}_g = \{\mathbf{g}_W, \mathbf{v}_W^{I_{\tau_0}}, \cdots, \mathbf{v}_W^{I_{\tau_{M-1}}}, \mathbf{b}_f, \mathbf{b}_\omega\}$, the corresponding optimization is

$$\tilde{\mathcal{S}}_g = \underset{\mathcal{S}_g}{\mathrm{argmin}} \sum_{i=0}^{M\text{-}2} \|\mathbf{r}_{I_i}\|^2, \tag{11}$$

with the residual $\mathbf{r}_{I_i}$ as

$$\mathbf{r}_{I_i} = \begin{bmatrix} \mathrm{Log}\left(\mathbf{R}_W^{I_{\tau_i}\top} \mathbf{R}_W^{I_{\tau_{i+1}}} \Delta\mathbf{R}_{\tau_i}^{\tau_{i+1}\top} \right) \\ \mathbf{v}_W^{I_{\tau_{i+1}}} - \mathbf{v}_W^{I_{\tau_i}} - \Delta\tau_i \mathbf{g}_W - \mathbf{R}_W^{I_{\tau_i}} \Delta\mathbf{v}_{\tau_i}^{\tau_{i+1}} \\ \mathbf{p}_W^{I_{\tau_{i+1}}} - \mathbf{p}_W^{I_{\tau_i}} - \Delta\tau_i \mathbf{v}_W^{I_{\tau_i}} - \frac{\Delta\tau_i}{2} \mathbf{g}_W - \mathbf{R}_W^{I_{\tau_i}} \Delta\mathbf{p}_{\tau_i}^{\tau_{i+1}} \end{bmatrix}. \tag{12}$$

Then, for any incoming undistorted scan and the associated linear body velocity $\mathbf{v}_{I_{\tau_i}}^{I_{\tau_i}}$, a measurement of the gravity $\mathbf{g}_{\tau_i}$ in the current frame is obtained as

$$\mathbf{g}\tau_i = \frac{\mathbf{v}_{I_{\tau_i}}^{I_{\tau_i}} - \Delta\mathbf{R}_{\tau_{i\text{-}1}}^{\tau_i\top} (\mathbf{v}_{I_{\tau_{i\text{-}1}}}^{I_{\tau_{i\text{-}1}}} - \Delta\mathbf{v}_{\tau_{i\text{-}1}}^{\tau_i})}{\Delta\tau_{i\text{-}1}}. \tag{13}$$

Finally, the gravity residual $\mathbf{r}_{g_i}$ is defined as

$$\mathbf{r}_{g_i} = \arccos\left(\frac{\mathbf{g}_W^\top \mathbf{R}_W^{I_{\tau_i}} \mathbf{g}_{\tau_i}}{\|\mathbf{g}_W\| \|\mathbf{g}_{\tau_i}\|} \right), \tag{14}$$

and its squared norm is added to the right-hand side of (10).

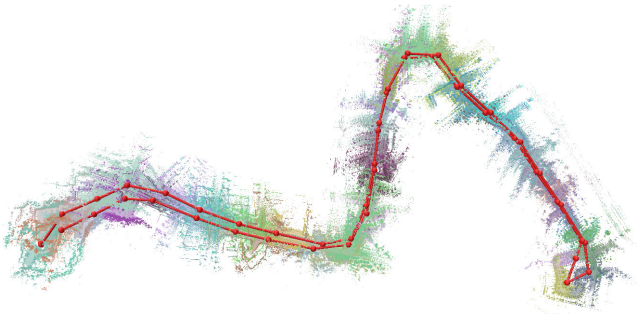


Figure 5. Example of topometric map obtained on a *Suburbs* sequence of the Boreas-RT dataset. It consists of a succession of submaps (shown as point clouds of different colours) and a topological graph (shown in red, with the sphere being the submaps' centroids) that connects the submaps. This topometric map does not require global consistency to enable state-of-the-art localization in repeating trajectories.

5.3 Topometric localization

Atop localization in a globally consistent map, 2Fast-2Lamaa enables topometric localization (and mapping) for repeated robot operations along the same path. This is inspired by the *Teach and Repeat* framework (Furgale and Barfoot 2010), which represents the environment with a graph of overlapping geometric submaps topologically linked based on a demonstrated robot trajectory. The localization process has two 'layers'. At the high level, it consists of moving from submap to submap (node to node) in the topological graph. At the low level, it performs scan-to-submap geometric registration. This approach alleviates the need for a globally consistent map while allowing high-precision localization for autonomous navigation. We have integrated this mapping and localization strategy into the proposed framework. In its current version, 2Fast-2Lamaa only considers the simplest type of graph: an *undirected chain*. Concretely, the localization in that chain consists of performing scan-to-submap registration using (10) and the submap associated with the current node in the chain. Then, if the state estimate reaches the edge of the submap, the next node (or previous one, depending on the direction of motion) becomes the current node. Fig. 5 illustrates a topometric map in the *Suburbs* environment of our self-collected dataset. While the trajectory used to create the map drifted from the ground-truth, the topometric nature of the localization enables accurate localization for repeating trajectories ($< 3\,\mathrm{cm}$ lateral position Root Mean Squared Error (RMSE), see Section 9.3).

6 Mapping

This section presents the proposed GP-based distance field for efficient scan-to-map registration in Section 5. The map can be provided with a single point cloud of the robot's environment or incrementally built according to the scan-to-map odometry estimates. The incremental map creation can be done using a single map or a collection of submaps for the topometric localization mode. The process presented in this section applies to all of these mapping strategies. The only difference for the topometric approach is that a new submap is created after a user-defined distance travelled since the last submap creation.

6.1 GP-based distance field

The proposed mapping process is based on GP-based distance fields (Le Gentil et al. 2024b). Using a point cloud as input, it performs standard GP regression to obtain a latent field $o(\mathbf{x})$ that can be seen as an occupancy field equal to 1 on object surfaces (where the lidar points lie), and that decreases to zero further away from it. Applying a specific non-linear function over $o(\mathbf{x})$ provides an approximation of the Euclidean distance between the query location $\mathbf{x}$ and the closest surface. Formally, o is modelled with a zero-mean GP

$$o(\mathbf{x}) \sim \mathcal{GP}\left(0, k\left(\mathbf{x}, \mathbf{x}'\right)\right), \tag{15}$$

with $k\left(\mathbf{x}, \mathbf{x}'\right)$ the covariance kernel function that specifies the covariance between two instances of the field $o(\mathbf{x})$ and $o(\mathbf{x}')$. Given a set of N observations/points (equal to one) at locations $\mathbf{X}$, and following standard GP regression (Rasmussen and Williams 2006), the mean and variance of the latent field can be inferred as

$$\begin{aligned} \hat{o}(\mathbf{x}) &= \mathbf{k}_{\mathbf{x},\mathbf{X}} \left(\mathbf{K}_{\mathbf{X},\mathbf{X}} + \sigma\mathbf{I}\right)^{-1} \mathbf{1}, \\ \text{var}(\hat{o}(\mathbf{x})) &= k\left(\mathbf{x}, \mathbf{x}\right) - \mathbf{k}_{\mathbf{x},\mathbf{X}} \left(\mathbf{K}_{\mathbf{X},\mathbf{X}} + \sigma\mathbf{I}\right)^{-1} \mathbf{k}_{\mathbf{x},\mathbf{X}}{}^{\top}, \end{aligned} \tag{16}$$

where $\mathbf{k}_{\mathbf{x},\mathbf{X}}$ is the vector of covariance kernels between the query point $\mathbf{x}$ and the input observation locations $\mathbf{X}$, $\mathbf{K}$, the kernel-based covariance matrix of $\mathbf{X}$, and σ the observation noise. Following Le Gentil et al. (2024b), the use of a 'reverting function' r over o approximates the Euclidean distance field

$$d(\mathbf{x}) = r\left(o(\mathbf{x})\right). \tag{17}$$

The definition of r depends on the kernel function k

$$r\left(k\left(\mathbf{x}, \mathbf{x}'\right)\right) \triangleq \|\mathbf{x} - \mathbf{x}'\|. \tag{18}$$

With an unscaled, stationary, isotropic kernel (that can be written as a function of the distance $k\left(\mathbf{x}, \mathbf{x}'\right) \rightarrow k(\|\mathbf{x} - \mathbf{x}'\|)$), the reverting function is the inverse of k. Note that due to the non-linear nature of r, the distance field d is not a GP. Accordingly, the inferred variance from (16) is not really informative about the distance estimate's uncertainty. While not used in 2Fast-2Lamaa, we propose a novel uncertainty proxy in Appendix A.1.

6.2 Efficient large-scale GP distance field

An issue with naively applying standard GP regression is the cubic computational complexity $O(N^3)$ with respect to the number N of points in the map due to the matrix inversion in (16). In this work, we propose a novel and efficient strategy for GP-based distance field by computing local GPs instead of a single global one. The core principle behind this strategy is the spatially limited impact of each input point due to the *lengthscale* of the kernel.* In other words, performing GP regression considering a large-enough local neighbourhood of points will (locally) yield a similar inferred mean when compared with using all the available observations.

The key to our approach is to leverage both a sparse voxelized representation $\mathcal{V}$ of the dense 3D point cloud of the environment, and a spatial index $\mathcal{I}$ that allows for efficient point insertion/removal and closest/radius neighbour searches.† The voxelized representation consists of a hashmap that maps 3-integer tuples to *cells*, which store the centroid of all the lidar/map points that occurred in the voxel. The tuples are the cell's grid index that can be computed directly from the 3D coordinates of a point and the fixed cell size.

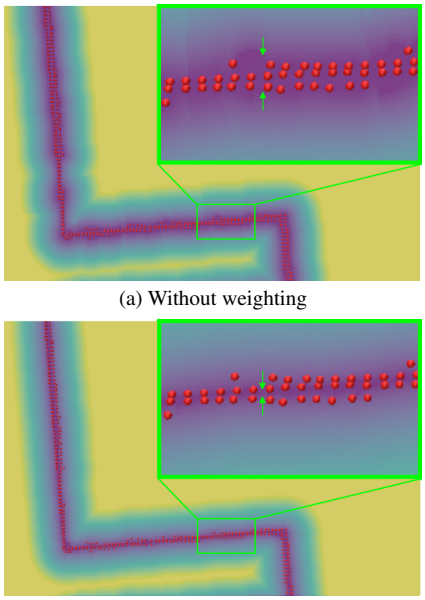
Integrating new points in the map simply consists of checking if the corresponding cell index is already present in the hashmap. If yes, the cell's centroid is updated incrementally with the new point location. Otherwise, a new cell is created, and its position is added to the spatial index. Thanks to the hashmap properties, accessing an existing cell or inserting a new one is constant-time $O(1)$ on average. The insertion of a new cell in a spatial index is slower with an $O(\log(N))$ average complexity at best. Fortunately, thanks to the voxelized nature of the stored information, only a small portion of the incoming lidar points lead to cell creation (for example, a 7.9 km-long suburban trajectory in the Boreas-RT dataset contains over 1.8 trillion lidar points, but the final number of cells in the map is around 10 million).

To query $d(\mathbf{x})$ at any location $\mathbf{x}$, the closest cell $\mathbf{v}_i$ in $\mathcal{V}$ is queried using the spatial index $\mathcal{I}$. Then, the local neighbourhood around $\mathbf{v}_i$ is obtained through a radius search in $\mathcal{I}$. This set of points represents the surface around the closest element to $\mathbf{x}$ in the map. An estimate of $d(\mathbf{x})$ can therefore be obtained using (16) and (17) with the aforementioned local neighbourhood as input observations. Considering the voxel nature of $\mathcal{V}$ and a fixed radius for the local neighbour search, there is a maximum of points M used for GP inference. Knowing that a query of $\mathcal{I}$ is $O(\log(N))$ on average, the overall distance field inference is $O(\log(N) + M^3)$. As M is fixed, the global complexity for large-scale mapping only grows logarithmically with the size of the observed environment.

There are two main caveats to the proposed field. The first is linked to the use of local GPs, creating a discrete behaviour when potentially switching from one local GP to the next while querying two locations that are side by side. In other words, the global field is not guaranteed to be continuous over the full space $\mathbb{R}^3$. However, the 'jumps' are generally small enough, in terms of value and gradient, that this does not prevent the use of the field in gradient-based optimization problems, as illustrated in this work. The second drawback is that the voxelized representation degrades the overall probabilistic properties of GP-based distance inference. Simply put, naively feeding the voxels' centroids to the GP inference using $\sigma\mathbf{I}$ as the measurement noise gives the same importance to each cell. However, not all cells provide the same amount of information: a cell in which only one point occurred should not have the same impact as a cell that averaged the position of 100 points. Fig. 6(a) shows the corresponding negative impact on the field: non-smooth surface and thicker walls. To alleviate this issue, we propose to 'weight' the cell centroid observations

*Most of the commonly used kernel functions rely on a lengthscale hyperparameter that characterizes the spread of the covariance function. An intuitive way to think about lengthscale is that it somewhat represents the maximum gap in observations that can be interpolated through GP regression.

†One can think about the spatial index as being a standard KD-Tree (Bentley 1975) however our implementation is based on a different data structure (cf. Section 8).



(a) Without weighting

(b) With proposed weighting

Figure 6. Illustration of the impact of the proposed weighting mechanism for our efficient GP-based distance field inference with sparse voxelized observations. The cell centroids are in red, and the colourmap represents the distance field (the colour is purposely saturated at 1m for the sake of visibility. (a) is the inference with equal weight for each voxel observation. (b) is the inference with the proposed weighting based on the number of lidar points that occurred in each cell. One can clearly see the improvement in terms of smoothness and 'wall thickness', thus the accuracy of the field.

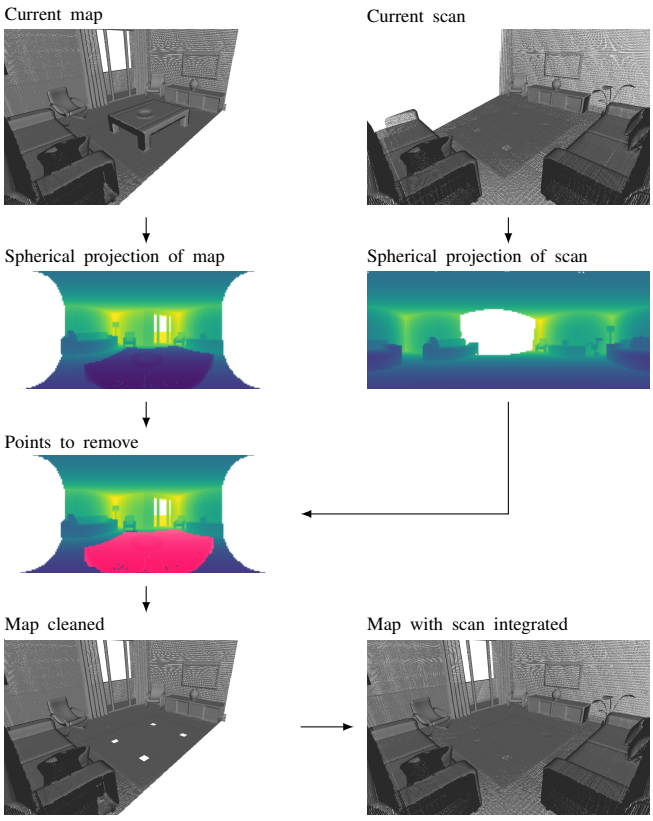


Figure 7. Illustration of the free-space carving process of the proposed mapping framework (using Handa et al. (2014)'s living room simulated environment).

based on the number of points that occurred in each voxel. Let us denote the per-voxel number of observations as the *counter* c_i of a cell. Accordingly, in (16), we replace the measurement uncertainty model $\sigma\mathbf{I}$ with $\mathrm{diag}(\mathbf{w})$ where w_i (components of $\mathbf{w}$) are computed as a function of the cell's counter and the maximum cell counter in the neighbourhood. Note that a lower w_i means that the cell can be trusted more than a cell with a higher w_i. Thus, the function that transforms c_i to w_i has to be decreasing. We picked a decreasing sigmoid function in the shape of $\frac{1}{1+\exp(-c_i/c_{\max})}$. Fig. 6(b) illustrates the improved efficient field inference with the proposed weighting mechanism. Appendix A.2 presents a brief quantitative analysis of the proposed efficient GP-based distance field with and without cell weighting.

6.3 Free-space carving

In self-driving scenarios (among others), it is impossible to guarantee that the environment is static at the time of mapping. Thus, a mechanism for rejecting dynamic objects from the map can help build cleaner maps for later reuse in a localization-only phase. 2Fast-2Lamaa includes a *free-space carving* step to remove dynamic points previously inserted in the map and account for changes in the environment, such as parked cars that are no longer present. The principle of the proposed mechanism relies on the comparison between the spherical projection of the current scan and that of the map cells in the vicinity of the current sensor position. First, the scan is projected onto an image-like data structure by discretizing the azimuth and elevation of each point using a resolution of around 1°. Each pixel retains the range value of the closest point that is projected into it. Then, the map points within a radius around the sensor position are queried and projected in the aforementioned image-like data structure using spherical coordinates. If the range of a map point is smaller than the corresponding scan range, using a margin equal to the voxel resolution, it is removed from the map as it occurred between a surface currently observed and the sensor. Fig. 7 illustrates this process with dense simulated data for the sake of clarity. Free-space carving can be performed online before inserting a freshly registered point cloud into the map, and offline by revisiting the whole trajectory and scans.

7 Online pose-graph optimization

When used in mapping mode with a single global map incrementally built based on odometry estimates, the proposed scan-to-map strategy cannot guarantee global consistency when the robot's trajectory forms loops due to the inherent drift in any odometry algorithm. While using submaps in a topometric approach alleviates this issue for any robotic application that involves repeated motion (teach-and-repeat-like), one might be interested in obtaining a globally consistent trajectory for globally consistent mapping. Toward this goal, 2Fast-2Lamaa also integrates

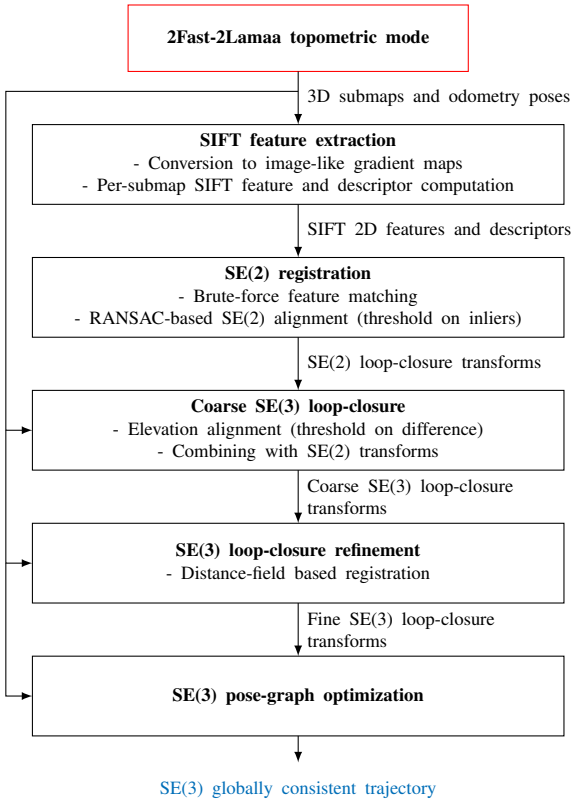
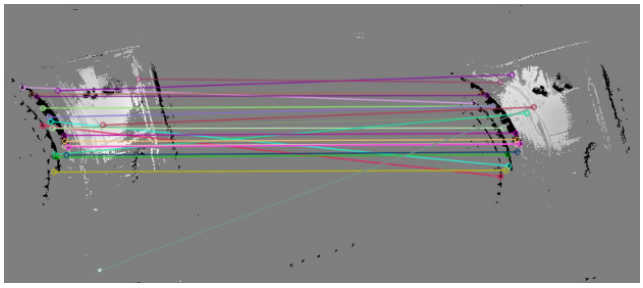
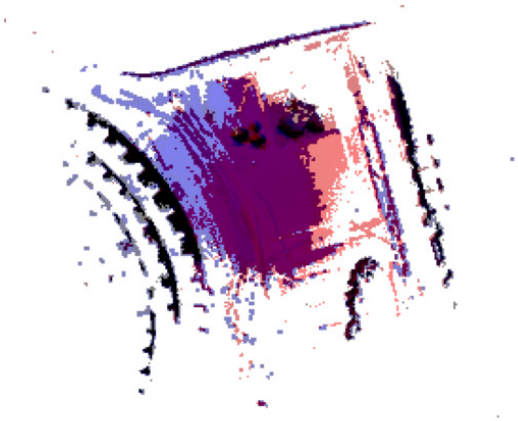


Figure 8. Overview of the proposed loop closure detection and correction based on the output of 2Fast-2Lamaa in topometric mapping mode. First, the 3D submaps are projected into image-like elevation gradient maps (2.5D). Using visual features, the 2.5D submaps are coarsely aligned by SE(2) RANSAC registration and elevation alignment. Thresholds on the number of inliers and the elevation coherence are used to discard non-loop-closure submap pairs. Based on the proposed GP-based distance field, coarse loop-closure transforms are refined before being used as residuals in a pose-graph optimization that leverages the odometry pose estimates between consecutive submaps.



(a) SIFT features matched between submaps



(b) Overlay after coarse SE(2) registration

Figure 9. Illustration of the loop-closure detection and SE(2) registration used in our online pose-graph optimization of 2Fast-2Lamaa's submaps. The proposed pipeline relies on the extraction, matching, and registration of visual features in image-like elevation representations of each submap.

an optional[‡] online loop-closure detection and correction mechanism based on feature descriptor matching and pose-graph optimization, similar to the work by Giubilato et al. (2022).

Fig. 8 shows the proposed pipeline. First, 2Fast-2Lamaa's topometric-mapping mode is leveraged to obtain submaps and submap-to-submap transformation estimates $\tilde{\mathbf{T}}_{s_i s_{i+1}}$. The voxelized representation of each gravity-aligned submap is projected in individual image-like data structures which correspond to the *elevation gradient* of the environment. Unlike the work by Giubilato et al. (2022), no GP is used to generate the image-like data. 2Fast-2Lamaa uses a simple top-down orthographic projection. The pixel size is one and a half times the voxel resolution. Then SIFT features (Lowe 1999) are extracted and matched between the current and past submaps as illustrated in Fig. 9. Loop-closure candidates are obtained by matching and registering features between submaps. To avoid considering wrong associations and limit the computational burden, the brute-force feature matching is only performed between submaps for which pose-to-pose distances are under a threshold computed based on the typical odometry drift in percentage (user-defined) and the distance travelled between pairs. Given the SIFT associations of a submap pair, an SE(2) transformation between the two submaps is estimated with RANSAC (Fischler and Bolles 1981) and elevation correction is performed so that the mean elevation is the same in both submaps. Combining these together, a rough loop-closure SE(3) transformation is obtained. Using the proposed continuous distance field (10), the SE(3) loop-closure constraint is refined individually between four scans of the oldest of the two matched submaps and the distance field of the current submap. Finally, the odometry relative pose estimates and the loop closure transformations are used in a pose-graph optimization. This non-linear least-squares problem is solved with the Levenberg-Marquardt algorithm.

8 Implementation

In this section, we present and discuss some specific details of our ROS2 C++ implementation.

8.1 Low-level data structures

The hashmaps in the work use the `ankerl::unordered_dense::map` implementation (Leitner-Ankerl 2022a). It has been benchmarked against numerous C++ hashmap implementations and displayed the best overall performance (Leitner-Ankerl 2022b). The

[‡]In our experiments, this feature is only used for evaluation on the VBR-SLAM benchmark.

cells are stored via pointers in the hashmap to ensure the compactness of the hashmap storage.

The constraints on the choice of the spatial index $\mathcal{I}$ are as follows:

- Fast and scalable insertion of new elements.
- Possibility to remove points without the need to recreate the index.
- Allowing for N-closest neighbour and radius search.

The ikd-Trees (Cai et al. 2021), PH-Trees (Zäschke et al. 2014), and i-Octrees (Zhu et al. 2024) match the aforementioned requirements. Appendix A.3 shows a toy-example evaluation. We observed a significant advantage for the i-Octree. For efficiency, the spatial indexing structure is not updated with the changing centroid of the voxels, but the hashmap is. The spatial index relies on the coordinates of the first point that occurred in each voxel. Accordingly, the closest neighbour or radius searches performed with the 'out-of-date' spatial index are only approximations. We found this not to be a problem empirically, as the GPs are computed with the up-to-date voxel centroids of the local neighbourhood that is more than twice as large as the voxel resolution. For some specific scenarios, such as queries in the middle of a corridor, the query for the closest voxel in the spatial index, whether it is out-of-date or up-to-date, can erroneously trigger GP computations using points from the furthest of the two sides. A simple workaround is to perform K closest neighbour searches with associated local GP-based distance field computation for each query. The final distance would be the smallest of the K resulting GP-inferred distances. Note that this corridor-like scenario is unlikely when performing scan-to-map registration thanks to the accurate initial guess from 2Fast-2Lamaa's undistortion module. Thus, our implementation uses $K = 1$ for registration. For other queries, the default is $K = 2$.

8.2 Non-linear optimizations

All the optimizations in this work (lidar-inertial undistortion and scan-to-map registration) are based on Ceres, an open-source non-linear least-squares solver (Agarwal and Mierle 2022). Both (5) and (10) leverage Cauchy loss functions to attenuate the impact of outliers. The analytical Jacobians of the residuals are provided to the solver. To lower the computational cost of the overall pipeline, we adopted a basic key-framing strategy for the scan-to-map registration, performing (10) only when the sensor has moved sufficiently or after a fixed time period.

8.3 GP kernel and distance field

While it was shown that the square-exponential provides the best accuracy for distance field estimation (Le Gentil et al. 2024b), 2Fast-2Lamaa's implementation uses the rational-quadratic kernel $(1 - \frac{\|\mathbf{x}-\mathbf{x}'\|^2}{2\alpha l^2})^{-\alpha}$ with $\alpha = 2$ for efficiency, as it avoids the use of operations such as exponential or logarithm, which are computationally costly. The lengthscale l is defined as twice the voxel resolution. To compute the local GP around a voxel, the neighbourhood search uses a radius of two and a half times the voxel resolution.

8.4 Point-cloud filtering

Lidar data is generally not affected by different levels of luminosity in the environment. However, it can be degraded by floating particles in the air, such as dust or precipitation. To increase robustness in more challenging scenarios, 2Fast-2Lamaa offers the possibility to filter incoming point clouds based on the sensor-provided per-point intensity. While not thoroughly analyzed in this paper, we have observed a significant improvement of 2Fast-2Lamaa's odometry and localization performance on the *Farm* sequences of the Boreas-RT dataset (Lisus et al. 2026), which were collected on an extremely dusty road. Additionally, a 'density check' is always performed on each undistorted point cloud in case the intensity filtering does not sufficiently remove amorphous point clusters. Using the user-provided map resolution, the scan is discretized into voxels. Points that fall into voxels that have more than twelve neighbours are ignored for registration.

8.5 Submapping strategy

When used with submaps, as opposed to building a single global map, 2Fast-2Lamaa uses a very simple submapping strategy: a scan is integrated in a submap if the sensor has travelled less than a user-defined distance γ since the submap creation. Note that the submaps overlap by a ratio β (equal to 0.2 in our experiments). Concretely, it means that a new submap is created when the distance travelled goes above $(1 - \beta)\gamma$. During the overlap, each scan is added to both submaps. When performing topometric localization, the change from one submap to the next is done when the current pose estimate is closer to the poses in the second half of the overlap than to the first half.

9 Experiments

We validate the proposed framework for both localization and odometry using various public datasets as follows: Section 9.2 uses the Boreas dataset; Section 9.3 is based on part of the Boreas-RT dataset; and Section 9.4 the Newer College dataset. We also leverage the VBR dataset for SLAM evaluation in Section 9.5. On top of this extensive benchmarking, we conduct an ablation using part of the Boreas-RT dataset in Section 9.6. Finally, Section 9.7 provides the reader with information about the computational requirements of 2Fast-2Lamaa.

9.1 Parameters

Aside from hardware-related variables such as extrinsic calibrations or maximum lidar range, 2Fast-2Lamaa can be tailored to specific applications or environments through a set of user-defined parameters. Table 1 provides an overview of the parameters that differ from dataset to dataset. Notably, the voxel size is smaller for the Newer College dataset, and a single global map is used as the scale of the environment is smaller than in other datasets, and the trajectory does not form large loops with loss of covisibility. As the VBR dataset contains some indoor and handheld sequences, we use a shorter submap length when compared to the Boreas datasets to increase the potential for submap-to-submap loop closures. Not that the VBR dataset evaluation is the

Table 1. List and description of the different parameters that differ between the different datasets

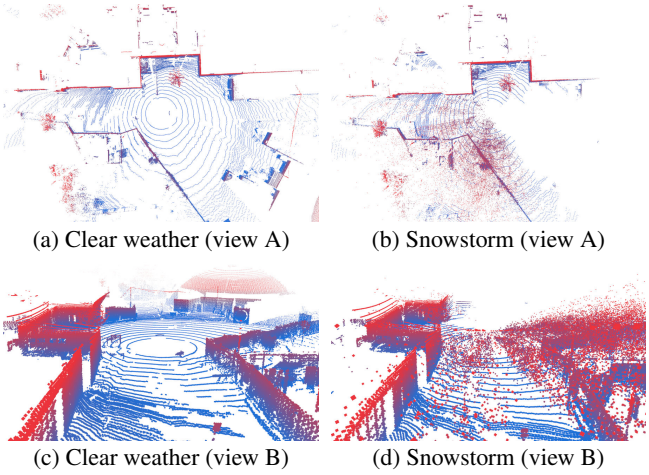
	Voxel size	Global/submaps	Keyframing	Num pts registration	Planar only	Free-space carving
Description	Size of the voxels for lidar feature extraction and map	Selection between the use of a single global map or submaps of given length γ	Travelled distance, orientation, and time threshold for keyframing scan-to-map registration	Maximum number of points for scan-to-map registration	Using only planar features	Radius for free-space carving
Boreas / Boreas-RT	0.30 m	Submaps 300 m	10 m/15°/0.5 s	8000	No	50 m
VBR	0.30 m	Submaps 50 m + online pose-graph	No	2000	No	50 m
Newer College	0.15 m	Global map	No	2000	Yes	No

only set of experiments that leverages the proposed loop-closure mechanism. As the Boreas and Boreas-RT datasets' IMU are of good quality, the optimization-based undistortion provides good odometry information. 2Fast-2Lamaa thus leverages a keyframing strategy to limit the number of scan-to-map registrations, allowing for the use of a higher number of points for registration. Finally, free-space carving is disabled in the Newer College dataset evaluation as very few dynamic objects are present in the scene. Please note that 2Fast-2Lamaa exposes many more parameters for the sake of generality, but only the ones mentioned here have been changed in our evaluation. Performance improvements could be attained by tuning parameters for each specific sequence type.

9.2 Boreas dataset

9.2.1 Dataset description: The Boreas dataset (Burnett et al. 2023) consists of 44 sequences collected along a single 8 km suburban route with an automotive sensing platform over the course of a year. The particularity of this dataset is the wide range of weather conditions in between different sequences, from bright sunny days to heavy rainfall and snowstorms. The vehicle is equipped with a Velodyne Alpha Prime lidar, a Navtech RAS6 imaging radar, a FLIR Blackfly S camera, and an Applanix RTK-GNSS-IMU ground-truthing solution. Fig. 10 illustrates the impact of extreme weather conditions on lidar data. The Boreas dataset withholds the ground-truth trajectory for 13 of the 44 sequences, corresponding to more than 100 km of data collection. These are used to benchmark various state estimation methods with a public leaderboard. While not optimal due to data contamination between the ground-truth solution and the input of the various algorithms, the Applanix IMU is used in conjunction with the lidar in the rest of that section, as it is the only inertial sensor present aboard the vehicle.

9.2.2 Baselines and metrics: For odometry, we report the leaderboard's KITTI odometry metric. Succinctly, this metric aligns trajectory chunks (every 10 lidar frames) of length $[100\,\text{m}, 200\,\text{m}, \ldots, 800\,\text{m}]$ based on the first pose of each chunk, computes the error in position and orientation at the end of each segment, and reports the average translation and rotation error relative to the distance travelled. By submitting our results to the leaderboard, we benchmark our method against LTR (Burnett et al. 2022), STEAM-LIO (Burnett et al. 2025a), and OG (Le Gentil et al. 2025). Both



(a) Clear weather (view A) (b) Snowstorm (view A)

(c) Clear weather (view B) (d) Snowstorm (view B)

Figure 10. Impact of a snowstorm on the lidar data in the Boreas dataset (sequences `2020-12-04-14-00` and `2021-01-26-10-59`). Ice has formed on the front-facing part of the lidar, completely blocking part of the sensor's FoV. Additionally, falling snow creates many points floating in the air (z-colored), making state estimation challenging.

LTR and STEAM-LIO are based on continuous-time ICP for lidar state estimation. The continuous trajectory formulation leverages efficient GPs with noise-on-acceleration motion priors. A major difference is the integration of inertial measurements in STEAM-LIO. The OG baseline does not use the lidar but solely the 3D gyroscope and wheel encoder. It represents the simplest and most efficient method for automotive odometry. Nonetheless, OG provides odometry accuracy on par with state-of-the-art exteroceptive frameworks.

To assess the localization ability of 2Fast-2Lamaa, we leverage the localization part of the leaderboard. It consists of creating a map of the route given one sequence, and localizing within that map using 10 other sequences. The metric is the RMSE of the relative pose between the trajectory of the mapping sequence and the localization ones. Note that only LTR has been submitted to the Boreas benchmark (more baselines are used with a subset of the Boreas-RT dataset in Section 9.3). LTR's strategy for localization is performing scan-to-local-map registration while accounting for past estimates and odometry in the form of a prior on the state.

9.2.3 Odometry benchmark: Table 2 reports the odometry accuracy of 2Fast-2Lamaa and the different baselines.

Table 2. Average relative pose accuracy of the proposed method and several baselines on the Boreas dataset leaderboard (Burnett et al. 2023).

Method	Trans. [%]	Rot. [°/100 m]
OG (Le Gentil et al. 2025)	0.54	**0.07**
LTR (Burnett et al. 2022)	0.54	0.16
STEAM-LIO (Burnett et al. 2025a)	0.45	0.15
2Fast-2Lamaa (ours)	**0.27**	0.10

Table 3. Per-sequence odometry error (KITTI metric) of 2Fast-2Lamaa on the Boreas dataset leaderboard.

Sequence ID	Weather conditions	Trans. err. [%]	Rot. err. [°/100 m]
2020-12-04-14-00		0.20	0.07
2021-01-26-10-59		0.36	0.14
2021-02-09-12-55		0.21	0.08
2021-03-09-14-23		0.26	0.10
2021-04-22-15-00		0.22	0.08
2021-06-29-18-53		0.28	0.10
2021-06-29-20-43		0.37	0.12
2021-09-08-21-00		0.31	0.11
2021-09-09-15-28		0.32	0.12
2021-10-05-15-35		0.30	0.10
2021-10-26-12-35		0.26	0.09
2021-11-06-18-55		0.25	0.09
2021-11-28-09-18		0.21	0.08

: Overcast, : Snow coverage, : High snow coverage
: Snowing, : Sun, : Rain, : Dusk, : Night

Except for OG, which displays a very small rotational error, 2Fast-2Lamaa is the most accurate method, significantly outperforming the other frameworks. In this experiment, 2Fast-2Lamaa uses submaps (topometric mapping) as it provides slightly better odometry results than the global map version on the Boreas-RT dataset. Section 9.6 provides more details about this difference. As shown in Table 3, 2Fast-2Lamaa's accuracy is consistent throughout the different sequences, except for `2021-01-26-10-59`, which has been collected in a snowstorm and ice accumulated on the front-facing part of the lidar, thus blocking a significant portion of the sensor's field of view (cf. Fig. 10). As all the methods provide decent odometry estimates, it is difficult to draw any strong conclusion on why our method performs better. Note that STEAM-LIO and LTR elegantly perform motion-distortion correction as part of their continuous-time state estimation without 'fixing' the undistortion in an open-loop fashion based on previous state estimates. Thus, the difference in the undistortion strategy is unlikely to explain the difference in the global performance. We believe that our continuous map representation makes a difference, as LTR and STEAM-LIO rely on discrete downsampled point clouds with normals to represent the environment. The impact of the continuous map is investigated in Section 9.6.

9.2.4 Localization benchmark: For consistency with the odometry benchmark and to match the topometric nature of LTR's maps, we used 2Fast-2Lamaa with submaps (as opposed to a single global map). 2Fast-2Lamaa's submaps correspond to 300 m of trajectory while LTR uses a threshold of 30 m to create a new submap. Table 4 provides the leaderboard results. LTR provides slightly better performance over most of the axes. We believe that the state prior in LTR's formulation helps stabilize the pose estimate between consecutive scans, thus providing more accurate localization abilities. This is supported by the higher roll and pitch rotational error of 2Fast-2Lamaa: our pure scan-to-map registration does not enforce any motion model nor leverage information from previous state estimates. Analyzing the per-sequence results for 2Fast-2Lamaa, there is no strong correlation between both the lateral and longitudinal error and the weather conditions. However, sequences with snow display a notably higher error along the vertical axis. This is expected as the mapping sequence `boreas-2020-11-26-13-58` is free from snow. Accordingly, the snow cover on the ground creates a vertical offset in the estimated poses. Overall, despite slightly worse performance compared to LTR, both methods offer high levels of accuracy with under 5 cm lateral RMSE, which is less than the width of any road lane marking.

9.3 Boreas-RT

9.3.1 Dataset description: To evaluate the proposed framework, we use data from the Boreas-RT dataset (Lisus et al. 2026). It uses the same sensing platform as the Boreas dataset with the main hardware differences being a different radar firmware and an additional 6-DoF Silicon Sensing DMU41 IMU to prevent data contamination between ground-truth and raw sensor data. In this section, we leverage 16 sequences in four different environments, totalling 120 km of data for benchmarking against various baselines. The different routes are *Suburbs*, *Regional*, *Tunnel*, and *Skyway*, listed in increasing level of difficulty. We also provide 2Fast-2Lamaa's results on all of the 60 sequences of Boreas-RT (≈650 km) in Appendix A.4.

Suburbs sequences follow the same route as in the Boreas dataset. The road traffic varies between sequences, but a lot of geometric features are present in the surroundings. The *Regional* data is collected at a higher speed with fewer geometric features to constrain the vehicle's pose. The *Tunnel* route goes through a kilometre-long tunnel with very few geometric features to leverage. Finally, the *Skyway* sequences go over a bridge/skyway with only a few lampposts as static features. A lot of the non-ground lidar points correspond to moving vehicles. Fig. 11 provides images from the onboard camera as well as the OpenStreetMap overlay of the trajectories. Both the *Suburbs* and *Skyway* routes form loops, whereas *Regional* and *Tunnel* sequences do not revisit previously driven areas. The *Regional* data present an extra challenge for localization: the route is driven twice in each direction, thus, when mapping with one sequence and localizing with the other three, two of the localization runs experience quite different viewpoints than the one of the mapping step (cf. Fig. 12). Similarly, the tunnel sequences are collected while driving in both directions. However, the tunnel consists of two separate tubes (one for each traffic direction) without co-visibility. Thus, our *Tunnel* localization experiments are run for each driving direction independently (two times: one sequence for

Table 4. RMSE localization error for 2Fast-2Lamaa and LTR on the Boreas dataset leaderboard.

Method		Long. [m]	Lat. [m]	Vert. [m]	Roll [°]	Pitch [°]	Yaw [°]
LTR (Burnett et al. 2022)		**0.039**	**0.031**	**0.055**	**0.043**	**0.025**	0.040
2Fast-2Lamaa topometric (ours)		0.048	0.046	0.061	0.080	0.037	**0.039**
↓ **2Fast-2Lamaa** per sequence							
2020-12-04-14-00		0.068	0.076	0.108	0.071	0.037	0.039
2021-01-26-10-59		0.040	0.030	0.132	0.103	0.093	0.038
2021-02-09-12-55		0.029	0.031	0.037	0.077	0.034	0.039
2021-03-09-14-23		0.037	0.029	0.045	0.079	0.034	0.031
2021-06-29-18-53		0.079	0.058	0.058	0.097	0.045	0.051
2021-09-08-21-00		0.078	0.062	0.065	0.103	0.042	0.049
2021-10-05-15-35		0.048	0.050	0.047	0.086	0.039	0.035
2021-10-26-12-35		0.057	0.059	0.047	0.087	0.043	0.043
2021-11-06-18-55		0.040	0.044	0.057	0.074	0.036	0.036
2021-11-28-09-18		0.043	0.033	0.074	0.075	0.038	0.032

: Overcast, : Snow coverage, : High snow coverage, : Snowing, : Sun, : Rain, : Dusk, : Night

mapping, one sequence for localization), and the quantitative results are the average of both.

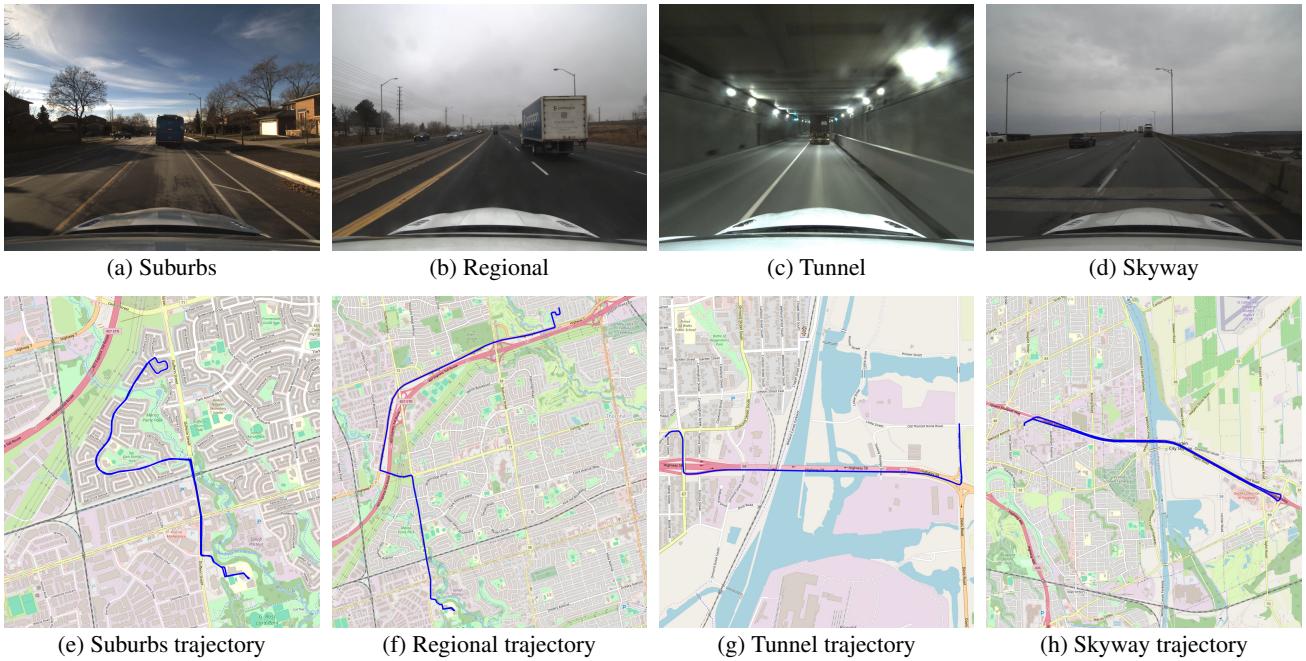
9.3.2 Baselines and metrics: The metrics used for odometry and localization are the same as for the Boreas experiments in Section 9.2. Regarding the odometry baselines, we benchmark 2Fast-2Lamaa against MOLA (Blanco-Claraco 2025), Fast-LIO2 (Xu et al. 2022), and LTR with gyroscope (Burnett et al. 2022). Each of these method address motion distortion in a different manner: MOLA assumes strict constant velocity during each lidar scan (optimizing for the velocity at each ICP step), Fast-LIO2 undistorts the incoming data based on the previous estimate and the open-loop propagation of the IMU readings, and LTR performs fully continuous-time estimation with GPs. Note that the LTR version here is different from the one present on the Boreas leaderboard. The main difference is the integration of angular velocity readings from the gyroscope as extra factors in the continuous-time optimization. For evaluating 2Fast-2Lamaa's localization abilities, we use LTR and Fast-LIO-localization (HViktorTsoi 2020) as baselines. The latter is an open-source project that performs localization in maps built with Fast-LIO2.

Sensor-specific parameters are updated for each baseline. Our experiments with MOLA use the default algorithmic parameters from the public implementation. Fast-LIO2 does not expose algorithmic parameters to the user. For LTR, the parameters have been tuned for the best average performance. Fast-LIO localization has been tuned to limit the number of critical failures.

9.3.3 Odometry benchmark: The odometry errors obtained with our method and the baselines are reported in Table 5. Both MOLA and Fast-LIO2 experienced failures in at least one sequence (large drift above $10\,\%$, or complete state divergence), with MOLA failing in half of the total sequences. For fairness, we only display the average across the successful runs. When both methods succeed in estimating the vehicle's trajectory, they reach a similar level of accuracy. LTR performs well when the environment is rich in geometric features (*Suburbs* and *Regional*), but its odometry error significantly increases for more challenging routes. 2Fast-2Lamaa is the only method that provides consistent results across all environments without failures. Its translation accuracy outperforms all the baselines. The *Suburbs* and *Regional* results of both LTR and 2Fast-2Lamaa highlight the accuracy gain of elegantly modelling the continuous motion of the sensors, thus addressing the problem of motion-distortion correction in a principled way. We believe that the increased difference in accuracy for more challenging sequences can be explained by the use of our continuous dense distance fields that better represent the environment.

9.3.4 Localization benchmark: Table 6 shows the results of our evaluation. On the *Suburbs* sequences, 2Fast-2Lamaa and LTR perform similarly with a slight advantage for LTR, as in the Boreas dataset. However, for all the other routes, 2Fast-2Lamaa is the best performing method with the smallest translational errors. When considering the more challenging environments (*Tunnel* and *Skyway*), the difference between our method and the baselines is larger. 2Fast-2Lamaa's error stays contained with RMSEs under $0.20\,\mathrm{m}$, while LTR and Fast-LIO-localization display errors of a couple of meters. Note that overall Fast-LIO-localization performs poorly and fails on many sequences (RMSE $> 10\,\mathrm{m}$). Fig. 13 shows the localization (position) for all methods and sequences as a function of the distance travelled. We believe that the Fast-LIO-localization implementation does not fuse the odometry information and the regular localization steps appropriately, and potential dropouts exacerbate the issue due to high computational load. This is supported by the noisy nature of Fast-LIO-localization's plots, which oscillate between centimetre- and meter-level errors. Eventually, most sequences end by drifting away from the original trajectory.

We also want to remind the reader that the *Regional* sequences are run in both directions for localization, while the mapping is done only in one, as previously illustrated in Fig. 12. When considering the sequence where the car is travelling in the same direction as the mapping run, the RMSEs are: Long. = $0.030\,\mathrm{m}$, Lat. = $0.025\,\mathrm{m}$, Vert. =



(a) Suburbs (b) Regional (c) Tunnel (d) Skyway

(e) Suburbs trajectory (f) Regional trajectory (g) Tunnel trajectory (h) Skyway trajectory

Figure 11. Images from the onboard camera of the Boreas-RT dataset and trajectory overlays (blue lines) on OpenStreetMap.

Table 5. Average relative pose accuracy of the proposed method and several baselines on a subset of the Boreas-RT dataset.

Sequence type (length, avg. / max. vel.)	**MOLA** (Blanco-Claraco 2025)	**Fast-LIO2** (Xu et al. 2022)	**LTR** w/ gyro (Burnett et al. 2022)	**2Fast-2Lamaa** topometric (ours)
Suburbs ($4 \times 7.9\,\mathrm{km}$, $8.1 / 18.6\,\mathrm{m/s}$)	0.74 / 0.27	0.74 / 0.20	0.29 / 0.09	**0.22** / **0.08**
Regional ($4 \times 9.3\,\mathrm{km}$, $11.1 / 27.0\,\mathrm{m/s}$)	0.81 / 0.23†	0.88 / 0.19*	0.35 / 0.09	**0.20** / **0.08**
Tunnel ($4 \times 1.9\,\mathrm{km}$, $8.9 / 28.2\,\mathrm{m/s}$)	1.72 / 0.30 †	0.83 / 0.17	2.49 / **0.09**	**0.34** / 0.11
Skyway ($4 \times 11.1\,\mathrm{km}$, $18.2 / 31.7\,\mathrm{m/s}$)	X†	0.98 / 0.20	0.59 / **0.08**	**0.30** / 0.09

KITTI odometry metric reported as *XX / YY* with *XX* [%] and *YY* [°/100 m] the translation and orientation errors, respectively.

† Mola displayed high drift (error $> 10\,\%$) on three sequences from Regional, one from Tunnel, and all from Skyway. Average computed by omitting these failed sequences.

* Fast-LIO2 failed on one of the Regional sequences (trajectory divergence). Average computed by omitting that sequence.

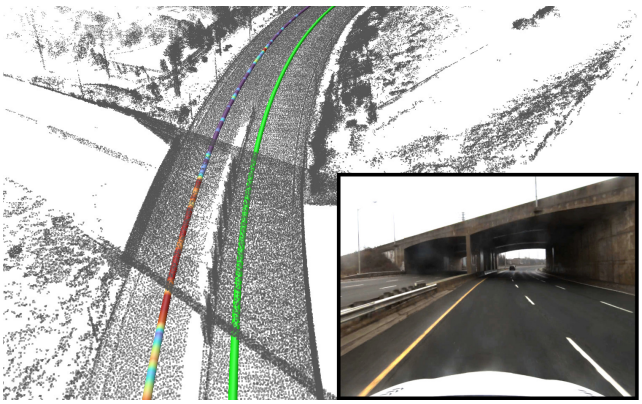


Figure 12. The four *Regional* sequences have been collected in both directions (two each way). When mapping with one sequence, part of the environment is only visible from the opposite traffic lane, thus leaving gaps in the map (as illustrated here under a bridge, with the map in gray, and the mapping trajectory in green). This makes localization challenging: the second line represents the localization trajectory of the second sequence and is colored with the localization error (purple low and red high).

$0.029\,\mathrm{m}$, Roll = $0.029°$, Pitch = $0.022°$, and Yaw = $0.028°$. These values are similar to those obtained in the easiest environment (*Suburbs*), demonstrating robustness to high-velocity trajectories.

9.4 Newer College dataset

9.4.1 Dataset description: To demonstrate the versatility of 2Fast-2Lamaa, we benchmark it using a dataset collected with a handheld device, the Newer College Dataset (Ramezani et al. 2020). It consists of 5 sequences collected while walking around the New College at the University of Oxford (UK) with a sensor suite equipped with an Intel RealSense D435i stereo camera and an Ouster OS1-64 lidar. Both sensors possess an embedded IMU. In our experiments, we only use the lidar and its internal IMU. The dataset also provides a detailed map of the environment as a coloured point cloud acquired with a Leica BLK360 survey lidar. The ground-truth trajectory of the sensors is obtained by registering the individual lidar scans to the map with ICP.

9.4.2 Baselines and metrics: For odometry, we adopt the standard Absolute Trajectory Error (ATE) to benchmark against various lidar-inertial frameworks. The ATE is defined as the RMSE between the positions of the estimated trajectory and the ones of the ground-truth after alignment using the Umeyama algorithm (Umeyama 1991). Similarly

Table 6. RMSE localization error for 2Fast-2Lamaa and different baselines on a subset of the Boreas-RT dataset.

Seq. type & Method	Long. [m]	Lat. [m]	Vert. [m]	Roll [°]	Pitch [°]	Yaw [°]
Suburbs						
Fast-LIO-localization (HViktorTsoi 2020)†	X	X	X	X	X	X
LTR w/ gyro (Burnett et al. 2022)	0.032	**0.021**	**0.032**	**0.013**	**0.011**	**0.021**
2Fast-2Lamaa topometric (ours)	**0.024**	**0.021**	0.034	0.036	0.025	0.026
Regional						
Fast-LIO-localization (HViktorTsoi 2020)†	2.031	0.510	0.603	0.310	0.660	1.183
LTR w/ gyro (Burnett et al. 2022)	0.056	0.055	0.061	**0.037**	0.039	0.035
2Fast-2Lamaa topometric (ours)	**0.042**	**0.035**	**0.029**	0.038	**0.036**	**0.033**
Tunnel						
Fast-LIO-localization (HViktorTsoi 2020)†	2.212	0.100	0.048	0.205	0.192	0.715
LTR w/ gyro (Burnett et al. 2022)	2.960	0.111	0.044	**0.063**	0.061	0.071
2Fast-2Lamaa topometric (ours)	**0.178**	**0.024**	**0.026**	0.087	**0.020**	**0.026**
Skyway						
Fast-LIO-localization (HViktorTsoi 2020)†	6.105	1.734	0.888	0.482	0.538	0.968
LTR w/ gyro (Burnett et al. 2022)*	7.502	0.203	0.085	0.054	0.042	0.175
2Fast-2Lamaa topometric (ours)	**0.059**	**0.026**	**0.035**	**0.050**	**0.030**	**0.028**

† Fast-LIO-localization succeeded (RMSE < 10 m) only on 2 out of 3 Regional sequences, 1 out of 2 Tunnel, and 1 out of 3 Skyway.

* Only 1 out of 3 Skyway sequences ran to completion for LTR.

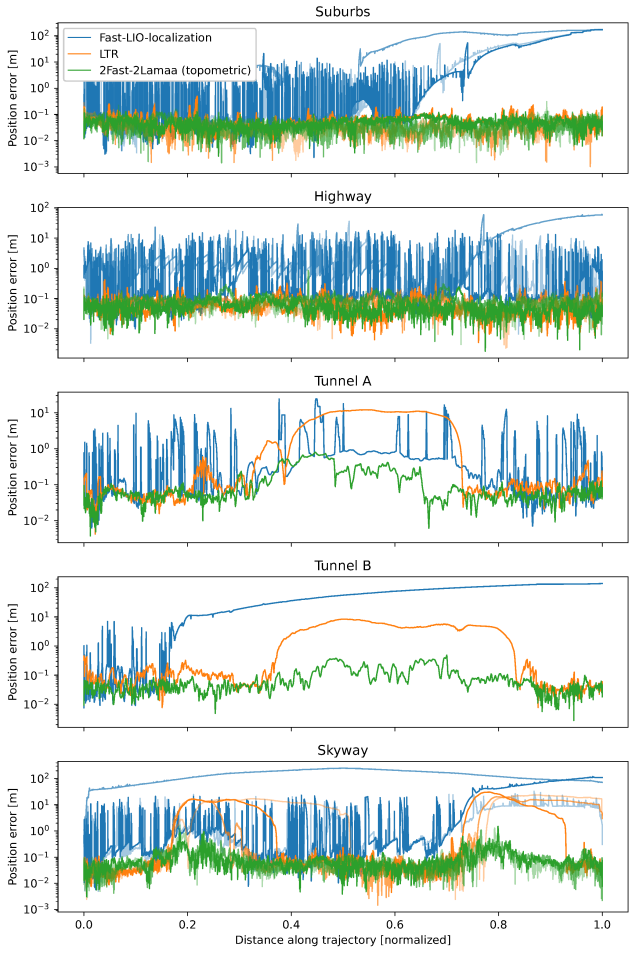


Figure 13. Localization error (position [m] with logarithmic scale) against travelled distance of 2Fast-2Lamaa and the different baselines on the Boreas-RT dataset. The different sequences are shown with different shades of colour.

to the evaluation with the Boreas-RT dataset, we use Fast-LIO2 (Xu et al. 2022) and STEAM-LIO (Burnett et al. 2025a) as baselines. We also include DLIO (Chen et al. 2023) as it is one of the best-performing methods on the Newer College dataset.

It is important to note that the ground-truth provided in the Newer College dataset is not as accurate as that of the other datasets in our experiments. In their paper, Ramezani et al. (2020) showed that when the system is static at the start of the dataset, the ground-truth position jitters in a sphere of around 15 cm in diameter. Additionally, we empirically found a non-negligible rotational error (up to 2°) that hinders the use of the Boreas localization metric, thus preventing the evaluation of localization from one sequence to the other.[§] Accordingly, we only perform localization against the provided map with 2Fast-2Lamaa as a demonstration of its ability to perform localization within an existing global map. We compute the localization RMSE directly between the provided ground-truth positions and the estimated ones as a sanity check.

9.4.3 Odometry benchmark: Table 7 shows the ATE obtained for each of the sequences. Note that the baseline results are the ones reported by Burnett et al. (2025a) for STEAM-LIO, and by Chen et al. (2023) for Fast-LIO2 and DLIO. Overall, all the methods provide a similar level of accuracy, with a slight advantage for 2Fast-2Lamaa for three out of the five sequences. This benchmark demonstrates that our method competes with state-of-the-art frameworks even when considering handheld sensor suites. It also shows that our method can be used with a low-cost IMU, such as the one embedded in the Ouster lidar.

[§] As a relative pose metric, the Boreas localization error is sensitive to ground-truth orientation accuracy. For example, with a localization run 10 m away from the mapping run, a ground-truth error of 1° leads to a position error of 17 cm due to the lever arm effect.

Table 7. ATE and RMSE localization error [m] for 2Fast-2Lamaa and various baselines on the Newer College dataset sequences.

Metric & Method	01_short (1609 m, 1530 s)	02_long (3063 m, 2180 s)	05_quad (479 m, 398 s)	06_spinning (97 m, 120 s)	07_parkland (696 m, 500 s)
ATE					
Fast-LIO2 (Xu et al. 2022)	0.378	0.332	0.088	0.078	0.148
DLIO (Chen et al. 2023)	0.360	0.327	0.084	**0.061**	0.120
STEAM-LIO (Burnett et al. 2025a)	0.304	0.337	0.109	0.082	0.144
2Fast-2Lamaa global map (ours)	**0.262**	**0.266**	**0.078**	0.085	**0.115**
Localization RMSE					
2Fast-2Lamaa global map (ours)	0.128	0.154	0.089	0.091	0.155

9.4.4 Localization benchmark: The localization RMSE of 2Fast-2Lamaa is reported in the last line of Table 7. As mentioned previously, the ground-truth accuracy level does not allow for a thorough analysis of the proposed pipeline's localization performance. Accordingly, it is difficult to draw any conclusion from these figures. However, these results demonstrate the soundness of the proposed approach and its ability to perform localization in a map generated with another sensor and mapping algorithm.

9.5 VBR-SLAM dataset

9.5.1 Dataset description: To further demonstrate the versatility of 2Fast-2Lamaa and the proposed online loop closure detection and correction mechanism, we benchmark our method on the VBR-SLAM dataset (Brizi et al. 2024). It consists of 16 sequences collected with a lidar, stereo vision, and an RTK-GPS-IMU solution in 6 different environments. Half of the sequences use an automotive platform with an Ouster OS0-64 lidar at 20 Hz (environments *Campus* and *Ciampino*). The other half is collected using a handheld sensor suite comprising an Ouster OS1-128 lidar, which collects scans at 10 Hz (environments *Colosseo*, *Pincio*, *Spagna*, and *DIAG*). In both scenarios, 2Fast-2Lamaa uses the lidar's embedded IMU. Out of the 16 sequences, only 8 have publicly available ground-truth. The ground-truth of the other sequences is held out for use in the dataset's public leaderboard. In this section, we only leverage sequences with provided ground-truth.[¶]

9.5.2 Baselines and metrics: As the trajectories form large loops, we use 2Fast-2Lamaa with the online loop-closure detection and correction mechanism from Section 6.3. We use PIN-SLAM (Pan et al. 2024), SMLE (Bhandari et al. 2024), and KISS-ICP (Vizzo et al. 2023) as lidar-based baselines. PIN-SLAM is especially designed for state estimation with loop closures. As done in the VBR-SLAM public benchmark, we report both the RMSE ATE, and the Relative Pose Error (RPE) as computed with the VBR-provided script. The RPE corresponds to the KITTI odometry metric, but computed with trajectory chunks of different sizes proportional to the whole trajectory length. The VBR-SLAM dataset also provides an overall score that aggregates the results throughout all the sequences to rank the different methods (the higher the better).

9.5.3 SLAM benchmark: Table 8 shows the results obtained with 2Fast-2Lamaa and the different baselines on both the training and testing sequences. While the ranking of the methods varies from one sequence to another, on average, our method displays the best performance, as shown by the overall score. Note that the second-best method, PIN-SLAM, requires a high-grade GPU to perform close to real-time operations. As discussed later in Section 9.7, 2Fast-2Lamaa only uses a fraction of a consumer-grade laptop CPU to achieve better performance. Our future work includes the formulation of a full-batch optimization built on point-to-surface residuals, with the proposed distance field, for better global map accuracy.

9.5.4 Odometry benchmark: For completeness, we also provide the RPE metrics as reported on the VBR-SLAM leaderboard in Table 9. SMLE displays the best performance on this benchmark. 2Fast-2Lamaa's results are respectively second and last on the training and testing sequences. It is important to note that the ground-truth of the VBR-SLAM dataset is obtained as a full-batch optimization of the lidar data using the RTK-GNSS measurements as priors. This approach does not provide an independent ground-truth and highly depends on the registration algorithm used. While sufficient for global trajectory consistency analysis (via the ATE), we believe it creates biases in the RPE evaluation. This latter metric is highly sensitive to the rotational accuracy of the ground-truth due to the lever effect of orientation inaccuracies when using long trajectory chunks as done in this leaderboard. For example, a rotation error of 0.1° corresponds to a translation error of around 0.17 m after 100 m travelled, which represents a minimum relative error of 0.17 %. To the best of our knowledge, at the time of writing, the VBR-SLAM ground-truth is computed without accounting for motion distortion. Accordingly, the rotational accuracy of the ground-truth is probably suboptimal, especially on handheld sequences where the sensors' orientation can change a lot during each lidar scan ($\gg 0.1^\circ$). This correlates with the fact that the best performing method, SMLE, does not perform motion distortion correction, and that 2Fast-2Lamaa's performance is significantly lower on the handheld sequences as opposed to the automotive ones.

9.6 Ablation study

2Fast-2Lamaa is a fairly modular framework. Thus, numerous combinations of components/features can be used

[¶]To avoid potential multiple submissions to the public leaderboard, due to software changes/improvements during the review process, we decided to submit our results to the leaderboard only the final version of the current paper.

Table 8. ATE obtained using the VBR-SLAM dataset.

Method	Colosseo [m]	Campus 0 [m]	Campus 1 [m]	Pincio [m]	Spagna [m]	DIAG [m]	Ciamp. 0 [m]	Ciamp. 1 [m]	Overall score
Training sequences									
PIN-SLAM (Pan et al. 2024)	**0.506**	**0.555**	**0.392**	0.647	0.480	0.362	1.366	1.253	74.44
SMLE (Bhandari et al. 2024)	1.411	1.196	0.994	0.793	0.696	0.457	1.495	1.308	71.65
KISS-ICP (Vizzo et al. 2023)	1.517	1.026	1.039	0.785	1.048	1.397	2.808	2.153	68.23
2Fast-2Lamaa pose-graph (ours)	0.634	0.900	0.584	**0.624**	**0.442**	**0.359**	**1.141**	**0.831**	**74.49**
Testing sequences									
PIN-SLAM (Pan et al. 2024)	0.422	0.570	**0.864**	1.063	0.373	**0.394**	1.790	1.521	73.00
SMLE (Bhandari et al. 2024)	0.454	0.897	1.022	1.071	**0.247**	0.456	1.350	**1.396**	73.11
KISS-ICP (Vizzo et al. 2023)	0.525	1.019	1.289	0.864	0.271	0.463	1.700	2.566	71.30
2Fast-2Lamaa pose-graph (ours)	**0.402**	**0.484**	0.874	**0.756**	0.350	0.405	**1.154**	1.558	**74.02**

Table 9. RPE obtained using the VBR-SLAM dataset.

Method	Colosseo [%]	Campus 0 [%]	Campus 1 [%]	Pincio [%]	Spagna [%]	DIAG [%]	Ciamp. 0 [%]	Ciamp. 1 [%]	Overall score
Training sequences									
PIN-SLAM (Pan et al. 2024)	0.490	**0.243**	**0.155**	**0.454**	11.61	0.468	0.738	0.648	66.80
SMLE (Bhandari et al. 2024)	**0.431**	0.613	0.312	0.461	**0.310**	**0.457**	**0.424**	**0.293**	**76.70**
KISS-ICP (Vizzo et al. 2023)	0.453	0.515	0.300	0.486	0.399	1.791	0.650	0.395	75.01
2Fast-2Lamaa pose-graph (ours)	0.905	0.599	0.390	0.821	0.581	0.718	0.525	0.372	75.09
Testing sequences									
PIN-SLAM (Pan et al. 2024)	**0.496**	**0.228**	**0.526**	1.082	0.456	0.550	1.018	0.769	74.88
SMLE (Bhandari et al. 2024)	**0.393**	0.251	0.639	0.557	**0.323**	**0.515**	**0.612**	**0.282**	**76.43**
KISS-ICP (Vizzo et al. 2023)	0.453	0.250	0.655	**0.488**	0.410	0.776	0.761	0.475	75.73
2Fast-2Lamaa pose-graph (ours)	0.969	0.356	0.703	0.932	0.701	1.025	0.653	0.629	74.03

Table 10. Ablation study of 2Fast-2Lamaa based on the Boreas-RT dataset and using the average translational KITTI odometry metric [%].

Variant	Suburbs	Regional	Tunnel	Skyway
Baseline	**0.22**	**0.20**	0.34	0.30
No edge	**0.22**	**0.20**	0.46	**0.26**
No field	0.29	0.24	0.35	0.31
Global map	0.27	**0.20**	0.32	0.32
No map	0.55	0.66	0.56	0.85
No carving	0.31	**0.20**	0.33	0.31†
No acc.	0.23	0.21	0.33	0.30
No IMU	0.25	0.25	0.36	0.33
No keyframe	0.25	0.26	**0.31**	0.36

† 2Fast-2Lamaa failed on one sequence.
Average computed with successful runs.

Table 11. Ablation study of 2Fast-2Lamaa based on the Boreas-RT dataset and using the RMSE position error [m].

Seq. type & Variant	Long.	Lat.	Vert.
Suburbs			
Baseline	**0.023**	**0.021**	**0.034**
Global map	0.024	0.022	**0.034**
Online-carved map	0.024	0.022	**0.034**
Uncarved map	0.024	0.022	**0.034**
Regional			
Baseline	**0.042**	0.035	0.029
Global map	0.044	**0.033**	**0.028**
Online-carved map	**0.042**	0.034	0.030
Uncarved map	**0.042**	**0.033**	0.029
Tunnel			
Baseline	0.178	**0.024**	0.026
Global map	0.216	0.025	**0.025**
Online-carved map	0.174	0.025	0.026
Uncarved map	**0.147**	0.025	**0.025**
Skyway			
Baseline	0.059	**0.026**	**0.035**
Global map	0.061	0.034	0.060
Online-carved map	0.057	**0.026**	0.036
Uncarved map	**0.052**	**0.026**	0.036

(global map vs. submaps, using edge features or not, cleaning the map or not, ...) to tailor the performance to a specific environment or application. In this section, we perform an ablation study using our 16-sequence subset of Boreas-RT (presented in Section 9.3). We refer to the full framework used to generate 2Fast-2Lamaa's results on Boreas and Boreas-RT datasets (Sections 9.2 and 9.3, respectively) as the 'baseline'. Each variant corresponds to a single difference from the baseline. All the results are presented in Tables 10

and 11 for the odometry and localization modes, respectively, and are discussed by theme in the following subsections.

9.6.1 Features: 2Fast-2Lamaa's undistortion modules extract planar and edge features from the raw data, and these are used for point-to-plane and point-to-line distance residuals in the optimization. As a reminder, the first type is obtained by simply subsampling the incoming data, while the second corresponds to jumps in range. As per the discrete nature of lidar data collection, limited by a certain angular resolution, the likelihood of acquiring a point exactly on the edge of an object in the real world is null. On the other hand, a point that belongs to a plane in the real world is not affected by the sensor resolution: multiple consecutive points are likely to belong to the same planar patch (given that the observed plane is large enough). Thus, when considering the reality of lidar data collection and the downstream distance-based residuals, edge features create noisier constraints in the optimization process. However, they provide more constraints due to a single DoF, where planar residuals have two. In environments that are rich with structural elements, we expect the use of both feature types to perform similarly or worse than using solely planar points. Inversely, edge features provide crucial complementary information in geometrically challenging environments such as tunnels, where planes alone do not constrain the longitudinal motion. This reasoning is empirically verified in Table 10 with the *No edge* variant that performs better than the baseline for *Skyway* and significantly worse on *Tunnel* sequences. Note that even without the edge features on these sequences, 2Fast-2Lamaa still outperforms the methods benchmarked in Table 5.

9.6.2 Continuous distance field: In this paragraph, we discuss the contribution of the continuous distance field map towards the global performance of the framework. 2Fast-2Lamaa allows for the direct use of point-to-point distances with the centroids of $\mathcal{V}$ in the scan-to-map registration step, instead of the GP-based distance field. The voxel size is the same as for the baseline underlying voxelized data structure: $0.3\,\mathrm{m}$ in all our automotive experiments. The corresponding *No field* variant in Table 10 shows a drop in performance with the relative translation error increasing by 20 to 30 percent in the more structured environments. While the difference is limited, using the proposed distance field has a non-negligible positive impact on the overall performance. Note that the planar motion in the *Skyway* sequences is mostly constrained by lampposts along the road. We believe that the fairly dense voxel-based data structure provides enough points along the posts, making the point-to-point residuals sufficient in effectively constraining the motion. Section 9.7 provides information on the difference in computation time between the baseline and the *No field* variant.

9.6.3 Global map vs. submaps: As discussed in Appendix A.3, the use of i-Octree enables fast distance queries in incrementally built large-scale maps. If the system's trajectory does not form large loops that accumulate too much drift, using a single map (as opposed to a succession of submaps) enables the estimation of globally consistent trajectories and maps. If the system's trajectory does not form large loops, the drift of the estimated state when revisiting previously mapped locations might

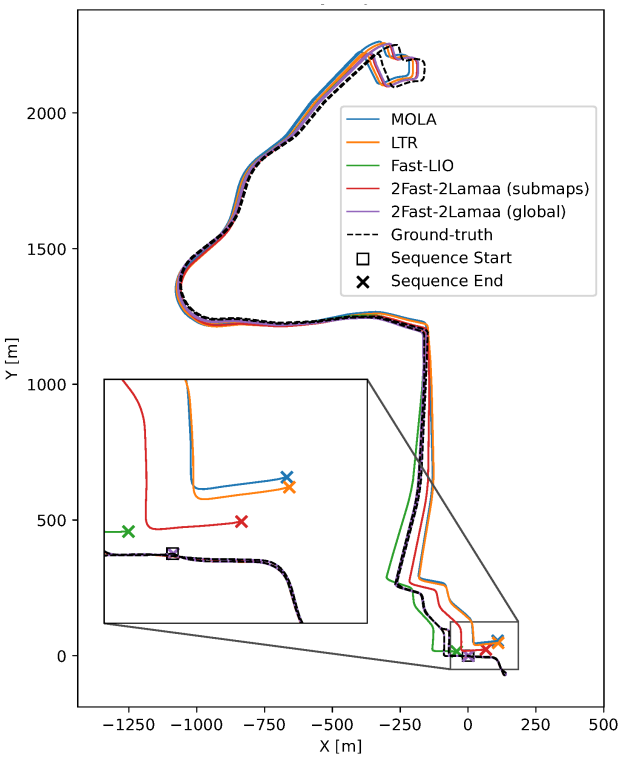


Figure 14. Estimated trajectories with 2Fast-2Lamaa and the different baselines on a *Suburbs* sequence. 2Fast-2Lamaa, with an incrementally built global map, is the only method that produces a trajectory that ends at the same location as the ground-truth. Note that none of the frameworks perform explicit loop-closure correction.

be small enough to enable the use of a single global map instead of a succession of submaps. Fig 14 illustrates the trajectory obtained when using an incrementally built global map on a *Suburbs* sequence. Note that 2Fast-2Lamaa is the only method that can provide a consistent trajectory (and map) as the other frameworks end up 'forgetting' previously traversed areas. Looking at the odometry metrics for *Global map* in Table 10, 'single-way' routes (*Regional* and *Tunnel*) are not affected by the change from submap to global map for scan-to-map registration, as expected. Interestingly, the routes that loop before heading back to the starting position are impacted negatively. This is due to the high sensitivity of the KITTI odometry metric to the orientation estimates. When the trajectory revisits a previously mapped area, due to some drift, the current state estimate can deviate from the locally consistent odometry to align the current data to the 'old part' of the map. This creates small 'jumps' in the trajectory estimate. As the KITTI odometry errors are computed after aligning a single pose per trajectory chunk, this deviation, especially in orientation due to a significant lever arm effect for trajectory chunks from 100 to $800\,\mathrm{m}$, locally degrades the metrics.

This last phenomenon does not really impact the localization performance within the global map in well-structured environments. As shown in Table 11, the results do not significantly change between the baseline and the *Global map* variant except for the *Tunnel* and *Skyway* sequences. We believe that the inherent drift of 2Fast-2Lamaa when building the global map hinders the sharpness of the few features present in the *Tunnel* and sequences (mostly corresponding

to sparse service doors). Having 'blurred' features in the map prevents high accuracy along the longitudinal axis. For the *Skyway* sequences, the vertical axis is significantly impacted, while the lateral error is slightly higher. This is due to the drift of the mapping trajectory creating 'double grounds and walls' in areas of the map. Note that the estimated pose errors stay similar for the other axes. While not really demonstrated here, an advantage of building globally consistent maps for later localization is the simplicity of the localization process. For 2Fast-2Lamaa, it is a simple scan-to-map registration. There is no need for a complex module to perform topometric graph navigation.

9.6.4 Map cleaning: Another component of our ablation study concerns the removal of dynamic objects from the submaps when performing odometry and localization. For odometry, the baseline includes an online free-space carving mechanism. The *No carving* variant does not possess this feature. The results in Table 10 show no difference on *Regional* and *Tunnel*, but the *Suburbs* and *Skyway* are negatively impacted, respectively with a higher error and a failure case.

As mentioned in the methodology part of this paper, the free-space carving can also be performed offline, given the mapping trajectory estimate, the map(s), and the undistorted scans. The localization baseline leverages both online and offline free-space carving during the mapping stage. The *Online-carved map* variant refers to the use solely of the online carving, and the *No carving* one does not perform any dynamic object removal. Fig. 15 illustrates the difference between the uncarved, online-carved, and offline-carved maps. Overall, the presence of dynamic points in the map does not seem to have a significant impact on localization on the Boreas-RT dataset. A deeper analysis of the impact of dynamic points for localization with more datasets and sensors is required before drawing any final conclusion.

9.6.5 IMU vs constant velocity 2Fast-2Lamaa's motion distortion correction is based on continuous preintegration to characterize the trajectory of the system. This continuous state can be replaced with a strict motion model during the interval $[\tau_{i\text{-}1}, \tau_{i+1}]$, similarly to MOLA. In this setup, we test two combinations of motion model: gyroscope preintegration with constant linear velocity (noted *No acc.*), and both linear and angular constant velocities (noted *No IMU*). The results in Table 10 show a drop in performance of both variants when compared to the baseline while still staying competitive. As expected, the performance of *No acc.* is better than that of *No IMU*. It is important to note that this analysis is based on automotive data. Thus, the movement of the platform is close to a constant velocity model. We expect a larger difference in performance with hand-held or drone-mounted sensors.

9.6.6 Keyframing To provide a deeper insight into our framework's parameters, we evaluate the 2Fast-2Lamaa's performance without keyframing and a lower number of points (2000) for scan-to-map registration, matching the parameters used on the VBR and Newer College datasets. This variant is denoted *No keyframe* in Table 10. Our method still outperforms state-of-the-art baselines from Table 5 on average. The total computational load is moderately impacted by this change of parameters, with $9.60\,\%$ total CPU load for the *Baseline* and $15.4\,\%$ for *No keyframe*.

Table 12. Computational load of various configurations of 2Fast-2Lamaa. The "core usage" corresponds to the average number of CPU cores used by the specific component.

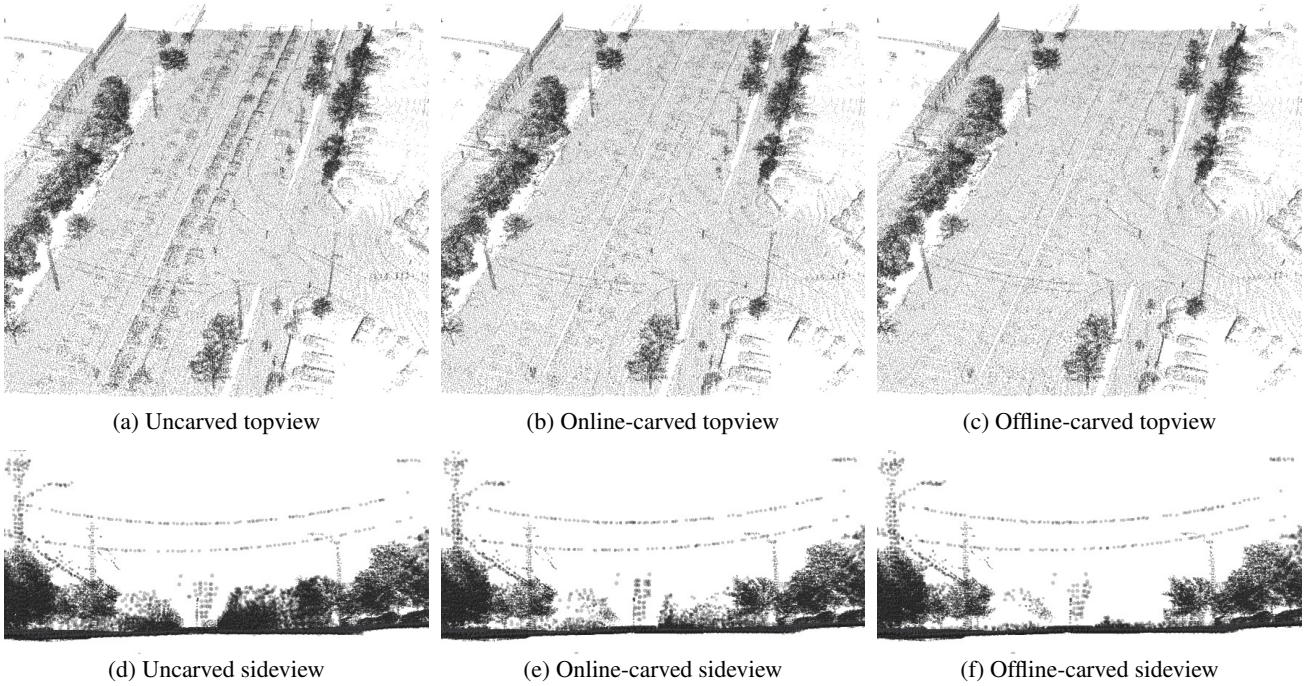
Variant	Undist. core usage	Loc. core usage	Total CPU [%]	Undist. RAM [GB]	Loc. RAM [GB]
Odometry					
Baseline	1.19	0.73	9.60	**0.17**	0.33
Global map	1.14	0.88	10.4	**0.17**	1.57
No field	1.09	**0.36**	**7.24**	**0.17**	**0.31**
No carving	**1.04**	0.59	8.12	0.19	0.33
No keyframe	1.42	1.66	15.4	0.18	0.50
Localization					
Baseline	1.06	**0.71**	**8.86**	0.17	**0.32**
Global map	**1.05**	0.78	9.16	**0.16**	2.24

9.7 Computational requirements

All the variants of 2Fast-2Lamaa presented in this paper run in real time on a laptop equipped with an Intel i7-1370p CPU and 32GB of RAM. Table 12 shows the CPU load and final RAM usage for a selection of variants from our ablation study. All the variants are lightweight and do not use more than $16\,\%$ of the CPU. Whether it is for odometry or localization, the main difference between using a global map or submaps resides in the larger RAM usage when using a global map. The similar CPU load in both scenarios highlights the efficiency of the i-Octree used for spatial indexing. Not using GPs or free-space carving results in a non-negligible drop in computational load. An even lighter version could be run by using neither of the two features. The *No keyframe* variant leads to the highest computational load. However, this is still moderate and leaves a lot of free CPU resources for other components in a standard robot navigation stack. In terms of computation time for the *Baseline*, on a typical *Suburbs* sequence, the undistortion/odometry optimization takes $41\,\mathrm{ms}$ on average, the scan-to-maps registration (for each keyframe) requires $61\,\mathrm{ms}$, and the subsequent map update lasts $54\,\mathrm{ms}$ with free-space carving and $19\,\mathrm{ms}$ without. Note that these numbers can be changed at will by changing the number of threads allocated to each one of the different modules. Note that the baselines (Fast-LIO2, MOLA, and LTR) all required a more powerful computer to run in real time (Intel i9-12900K).

10 Conclusion

In this paper, we present 2Fast-2Lamaa, a lidar-inertial mapping and localization framework that consists of two main components: an optimization-based motion-distortion correction module and a scan-to-map localization algorithm that can build maps incrementally. Given sufficient geometric features in the lidar's surroundings, the undistortion step estimates the continuous trajectory of the sensor through point-to-line and point-to-plane distance minimization between consecutive scans. The local trajectory is characterized by the inertial data directly



(a) Uncarved topview (b) Online-carved topview (c) Offline-carved topview

(d) Uncarved sideview (e) Online-carved sideview (f) Offline-carved sideview

Figure 15. Illustration of the dynamic object removal results in a map slice from a *Suburbs* sequence. Most of the dynamic objects are removed by the online free space carving process.

through continuous IMU preintegration. Thus, the initial conditions (gravity direction, linear velocity, and biases) are the only state variables to estimate, making the problem efficient to solve. For motion consistency over longer horizons, the scan-to-map registration is performed using GP-based distance fields. Thanks to the use of efficient data structures and locally computed GPs, the scan alignment can be done in real time. In odometry/mapping mode, the map is incremented after every scan-to-map registration. An optional online loop-closure detection and correction step, based on a pose-graph optimization, gives 2Fast-2Lamaa the ability to estimate globally consistent trajectories if required.

Throughout an extensive experimental analysis using data from automotive and handheld platforms, 2Fast-2Lamaa demonstrates state-of-the-art accuracy for odometry, localization, and SLAM. Results were consistent across the different test environments, even with challenging routes going through feature-limited tunnels where baselines' performance significantly declined. Note that these environments are not fully feature-deprived, as regular features such as service doors are present along the traffic lane. To address fully feature-deprived scenarios, our future work includes three different directions that are: the integration of intensity information in both the undistortion and localization modules, the fusion with vision sensing, and the exploration of Doppler velocity measurement with Frequency Modulated Continuous Wave (FMCW) lidars.

References

Agarwal S and Mierle K (2022) Ceres Solver. URL http://ceres-solver.org.

Barfoot TD (2024) *State estimation for robotics*. Second edition. Cambridge University Press.

Baumgartner E and Skaar S (1994) An autonomous vision-based mobile robot. *IEEE Trans. on Automatic Control* 39(3): 493–502. DOI:10.1109/9.280748.

Bentley JL (1975) Multidimensional binary search trees used for associative searching. *Communications of the ACM* 18(9): 509–517. DOI:10.1145/361002.361007.

Besl P and McKay ND (1992) A method for registration of 3-d shapes. *IEEE Trans. on Pattern Analysis and Machine Intelligence (TPAMI)* 14(2): 239–256. DOI:10.1109/34.121791.

Bhandari V, Phillips TG and McAree PR (2024) Minimal configuration point cloud odometry and mapping. *Int. Journal of Robotics Research* 43(11): 1831–1850. DOI:10.1177/02783649241235325.

Biber P and Strasser W (2003) The normal distributions transform: a new approach to laser scan matching. In: *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, volume 3. pp. 2743–2748 vol.3. DOI:10.1109/IROS.2003.1249285.

Blanco-Claraco JL (2025) A flexible framework for accurate LiDAR odometry, map manipulation, and localization. *Int. Journal of Robotics Research* 44(9): 1553–1599. DOI:10.1177/02783649251316881.

Boche S, Jung J, Laina SB and Leutenegger S (2025) Okvis2-x: Open keyframe-based visual-inertial slam configurable with dense depth or lidar, and gnss. *IEEE Trans. on Robotics (TRO)* 41: 6064–6083. DOI:10.1109/TRO.2025.3619051.

Boche S, Laina SB and Leutenegger S (2024) Tightly-coupled lidar-visual-inertial slam and large-scale volumetric occupancy mapping. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. pp. 18027–18033. DOI:10.1109/ICRA57147.2024.10610460.

Bosse M, Zlot R and Flick P (2012) Zebedee : Design of a spring-mounted 3-D range sensor with application to mobile mapping. *IEEE Trans. on Robotics (TRO)* 28(October): 1–15.

Brizi L, Giacomini E, Giammarino LD, Ferrari S, Salem O, Rebotti LD and Grisetti G (2024) Vbr: A vision benchmark in rome. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. pp. 15868–15874. DOI:10.1109/ICRA57147.2024.10611395.

Brooks RA (1987) *Visual map making for a mobile robot*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc. ISBN 0934613338, p. 438–443.

Burnett K, Schoellig AP and Barfoot TD (2025a) Continuous-time radar-inertial and lidar-inertial odometry using a gaussian process motion prior. *IEEE Trans. on Robotics (TRO)* 41: 1059–1076. DOI:10.1109/TRO.2024.3521856.

Burnett K, Schoellig AP and Barfoot TD (2025b) Imu as an input versus a measurement of the state in inertial-aided state estimation. *Robotica* 43(2): 680–700. DOI:10.1017/S0263574724002121.

Burnett K, Wu Y, Yoon DJ, Schoellig AP and Barfoot TD (2022) Are We Ready for Radar to Replace Lidar in All-weather Mapping and Localization? *IEEE Robotics and Automation Letters (RA-L)* 7(4): 10328–10335.

Burnett K, Yoon DJ, Wu Y, Li AZ, Zhang H, Lu S, Qian J, Tseng WK, Lambert A, Leung KY, Schoellig AP and Barfoot TD (2023) Boreas: A multi-season autonomous driving dataset. *Int. Journal of Robotics Research* 42(1-2): 33–42.

Cai Y, Xu W and Zhang F (2021) ikd-tree: An incremental kd tree for robotic applications. *arXiv preprint* .

Cao Z, Talbot W and Li K (2025) Resple: Recursive spline estimation for lidar-based odometry. *IEEE Robotics and Automation Letters (RA-L)* 10(10): 10666–10673. DOI:10.1109/LRA.2025.3604758.

Chen K, Nemiroff R and Lopez BT (2023) Direct lidar-inertial odometry: Lightweight lio with continuous-time motion correction. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. pp. 3983–3989. DOI:10.1109/ICRA48891.2023.10160508.

Chen Z, Le Gentil C, Lin F, Lu M, Qiao Q, Xu B, Qi Y and Lu P (2025) Breaking the static assumption: A dynamic-aware lio framework via spatio-temporal normal analysis. *IEEE Robotics and Automation Letters (RA-L)* 10(12): 12636–12643. DOI:10.1109/LRA.2025.3623436.

Cioffi G, Cieslewski T and Scaramuzza D (2022) Continuous-time vs. discrete-time vision-based slam: A comparative study. *IEEE Robotics and Automation Letters (RA-L)* 7(2): 2399–2406. DOI:10.1109/LRA.2022.3143303.

Coto-Elena L, Maese J, Merino L and Caballero F (2026) D-lio: 6dof direct lidar-inertial odometry based on simultaneous truncated distance field mapping. *IEEE Robotics and Automation Letters (RA-L)* 11(1): 169–176. DOI:10.1109/LRA.2025.3632615.

Dellenbach P, Deschaud JE, Jacquet B and Goulette F (2022) Ct-icp: Real-time elastic lidar odometry with loop closure. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. pp. 5580–5586. DOI:10.1109/ICRA46639.2022.9811849.

Deng J, Wu Q, Chen X, Xia S, Sun Z, Liu G, Yu W and Pei L (2023) Nerf-loam: Neural implicit representation for large-scale incremental lidar odometry and mapping. In: *Proc. of the IEEE Intl. Conf. on Computer Vision (ICCV)*.

Dissanayake MG, Newman P, Clark S, Durrant-Whyte HF and Csorba M (2001) A solution to the simultaneous localization and map building (slam) problem. *IEEE Transactions on robotics and automation* 17(3): 229–241.

Droeschel D and Behnke S (2018) Efficient Continuous-time SLAM for 3D Lidar-based Online Mapping. *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)* (May): 5000–5007. DOI:10.0/Linux-x86_64.

Duberg D, Zhang Q, Jia M and Jensfelt P (2024) Dufomap: Efficient dynamic awareness mapping. *IEEE Robotics and Automation Letters (RA-L)* 9(6): 5038–5045. DOI:10.1109/LRA.2024.3387658.

Eckenhoff K, Geneva P and Huang G (2019) Closed-form preintegration methods for graph-based visual–inertial navigation. *Int. Journal of Robotics Research* 38(5): 563–586.

Falque R, Le Gentil C and Sukkar F (2023) Dynamic object detection in range data using spatiotemporal normals. In: *Proc. of the Australasian Conf. on Robotics and Automation (ACRA)*.

Fischler MA and Bolles RC (1981) Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM* 24(6): 381–395.

Forster C, Carlone L, Dellaert F and Scaramuzza D (2017) On-Manifold Preintegration for Real-Time Visual-Inertial Odometry. *IEEE Trans. on Robotics (TRO)* 33(1): 1–21.

Funk N, Tarrio J, Papatheodorou S, Popović M, Alcantarilla PF and Leutenegger S (2021) Multi-resolution 3D mapping with explicit free space representation for fast and accurate mobile robot motion planning. *IEEE Robotics and Automation Letters (RA-L)* 6(2): 3553–3560.

Furgale P and Barfoot TD (2010) Visual teach and repeat for long-range rover autonomy. *Journal of Field Robotics (JFR)* 27(5): 534–560.

Giubilato R, Le Gentil C, Vayugundla M, Schuster MJ, Vidal-Calleja T and Triebel R (2022) Gpgm-slam: a robust slam system for unstructured planetary environments with gaussian process gradient maps. *Field Robotics* 2: 1721–1753.

Guadagnino T, Mersch B, Gupta S, Vizzo I, Grisetti G and Stachniss C (2025) Kiss-slam: A simple, robust, and accurate 3d lidar slam system with enhanced generalization capabilities. In: *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. pp. 5363–5370. DOI:10.1109/IROS60139.2025.11246613.

Ha S, Yeon J and Yu H (2024) Rgbd gs-icp slam. In: *Proc. of the Europ. Conf. on Computer Vision (ECCV)*. Berlin, Heidelberg: Springer-Verlag. ISBN 978-3-031-72763-4, p. 180–197. DOI:10.1007/978-3-031-72764-1_11.

Han L, Gao F, Zhou B and Shen S (2019) Fiesta: Fast incremental euclidean distance fields for online motion planning of aerial robots. In: *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. IEEE, pp. 4423–4430.

Handa A, Whelan T, McDonald J and Davison AJ (2014) A benchmark for rgb-d visual odometry, 3d reconstruction and slam. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. pp. 1524–1531. DOI:10.1109/ICRA.2014.6907054.

Hatleskog J, Nissov M and Alexis K (2025) Imu-preintegrated radar factors for asynchronous radar-lidar-inertial slam. In: *Advanced Robotics*.

Huang GP, Le Gentil C, Vidal-Calleja T, Scaramuzza D, Dellaert F and Carlone L (2026) Inertial odometry for SLAM. In:

Carlone L, Kim A, Barfoot T, Cremers D and Dellaert F (eds.) *SLAM Handbook. From Localization and Mapping to Spatial Intelligence*. Cambridge University Press.

HViktorTsoi (2020) Fast-LIO localization: A simple localization framework that can re-localize in built maps based on Fast-LIO. — github.com. https://github.com/HViktorTsoi/FAST_LIO_LOCALIZATION.

Jia M, Zhang Q, Yang B, Wu J, Liu M and Jensfelt P (2024a) Beautymap: Binary-encoded adaptable ground matrix for dynamic points removal in global maps. *IEEE Robotics and Automation Letters (RA-L)* 9: 6256–6263.

Jia Y, Wang T, Chen X and Shao S (2024b) Trlo: An efficient lidar odometry with 3d dynamic object tracking and removal. *arXiv preprint* .

Le Gentil C, Falque R and Vidal-Calleja T (2024a) Real-time truly-coupled lidar-inertial motion correction and spatiotemporal dynamic object detection. In: *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. IEEE, pp. 12565–12572.

Le Gentil C, Lisus D and Barfoot TD (2025) Do we still need to work on odometry for autonomous driving? In: *International Conference on Robotics and Automation 2025 Workshop on Field Robotics*.

Le Gentil C, Ouabi OL, Wu L, Pradalier C and Vidal-Calleja T (2024b) Accurate gaussian-process-based distance fields with applications to echolocation and mapping. *IEEE Robotics and Automation Letters (RA-L)* 9(2): 1365–1372.

Le Gentil C, Tschopp F, Alzugaray I, Vidal-calleja T, Siegwart R and Nieto J (2020a) IDOL : A Framework for IMU-DVS Odometry using Lines. In: *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. pp. 5863–5870.

Le Gentil C and Vidal-Calleja T (2023) Continuous latent state preintegration for inertial-aided systems. *Int. Journal of Robotics Research* 42(10): 874–900.

Le Gentil C, Vidal-Calleja T and Huang S (2018) 3d lidar-imu calibration based on upsampled preintegrated measurements for motion distortion correction. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. pp. 2149–2155. DOI:10.1109/ICRA.2018.8460179.

Le Gentil C, Vidal-Calleja T and Huang S (2020b) Gaussian Process Preintegration for Inertial-Aided State Estimation. *IEEE Robotics and Automation Letters (RA-L)* 5(2): 2108–2114.

Le Gentil C, Vidal-Calleja T and Huang S (2021) In2laama: Inertial lidar localization autocalibration and mapping. *IEEE Trans. on Robotics (TRO)* 37(1): 275–290. DOI:10.1109/TRO.2020.3018641.

Lee D, Jung M, Yang W and Kim A (2024) Lidar odometry survey: recent advancements and remaining challenges. *Intelligent Service Robotics* : 1–24.

Leitner-Ankerl M (2022a) A fast & densely stored hashmap and hashset based on robin-hood backward shift deletion. URL https://github.com/martinus/unordered_dense.

Leitner-Ankerl M (2022b) Comprehensive C++ Hashmap Benchmarks 2022. URL https://martin.ankerl.com/2022/08/27/hashmap-bench-01/.

Li X, Wang Z, Liu Z, Zhang Y, Zhang F, Yao X and Huang P (2024) Asynchronous event-inertial odometry using a unified gaussian process regression framework. In: *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. pp. 7773–7778. DOI:10.1109/IROS58592.2024.10802357.

Lichtenfeld J, Daun K and von Stryk O (2024) Efficient dynamic lidar odometry for mobile robots with structured point clouds. In: *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. IEEE, pp. 10137–10144.

Lilge S and Barfoot TD (2025) Incorporating control inputs in continuous-time gaussian process state estimation for robotics. *Robotica* 43(3): 1067–1086. DOI:10.1017/S0263574725000116.

Lin J, Yuan C, Cai Y, Li H, Ren Y, Zou Y, Hong X and Zhang F (2023) Immesh: An immediate lidar localization and meshing framework. *IEEE Trans. on Robotics (TRO)* 39(6): 4312–4331. DOI:10.1109/TRO.2023.3321227.

Lisus D, Papais KM, Le Gentil C, Preston-Krebs E, Lambert A, Leung KYK and Barfoot TD (2026) Boreas road trip: A multi-sensor autonomous driving dataset on challenging roads. *The International Journal of Robotics Research* 0(0). DOI:10.1177/02783649261479362.

Lopez BT (2023) A contracting hierarchical observer for pose-inertial fusion. *arXiv preprint* .

Lowe DG (1999) Object recognition from local scale-invariant features. In: *Proc. of the IEEE international conference on computer vision*, volume 2. Ieee, pp. 1150–1157.

Lupton T and Sukkarieh S (2012) Visual-inertial-aided navigation for high-dynamic motion in built environments without initial conditions. *IEEE Trans. on Robotics (TRO)* 28(1): 61–76.

Magnusson M (2009) *The three-dimensional normal-distributions transform: an efficient representation for registration, surface analysis, and loop detection*. PhD Thesis, Örebro universitet.

Museth K (2013) Vdb: High-resolution sparse volumes with dynamic topology. *ACM Transactions on Graphics* .

Museth K, Lait J, Johanson J, Budsberg J, Henderson R, Alden M, Cucka P, Hill D and Pearce A (2013) Openvdb: an open-source data structure and toolkit for high-resolution volumes. In: *Acm siggraph 2013 courses*. pp. 1–1.

Noh C, Yang W, Jung M, Jung S and Kim A (2025) Garlio: Gravity enhanced radar-lidar-inertial odometry. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. pp. 9869–9875. DOI:10.1109/ICRA55743.2025.11128334.

Oleynikova H, Taylor Z, Fehr M, Siegwart R and Nieto J (2017) Voxblox: Incremental 3d euclidean signed distance fields for on-board mav planning. In: *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*.

Ortiz J, Clegg A, Dong J, Sucar E, Novotny D, Zollhoefer M and Mukadam M (2022) isdf: Real-time neural signed distance fields for robot perception. In: *Robotics: Science and Systems*.

Pan Y, Zhong X, Jin L, Wiesmann L, Popović M, Behley J and Stachniss C (2025) PINGS: Gaussian Splatting Meets Distance Fields within a Point-Based Implicit Neural Map. In: *Proc. of Robotics: Science and Systems (RSS)*.

Pan Y, Zhong X, Wiesmann L, Posewsky T, Behley J and Stachniss C (2024) PIN-SLAM: LiDAR SLAM Using a Point-Based Implicit Neural Representation for Achieving Global Map Consistency. *IEEE Trans. on Robotics (TRO)* 40: 4045–4064.

Park C, Moghadam P, Kim S, Elfes A, Fookes C and Sridharan S (2018) Elastic LiDAR Fusion: Dense Map-Centric Continuous-Time SLAM. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*.

Park JJ, Florence P, Straub J, Newcombe R and Lovegrove S (2019) Deepsdf: Learning continuous signed distance functions for shape representation. In: *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*.

Pfreundschuh P, Hendrikx HF, Reijgwart V, Dubé R, Siegwart R and Cramariuc A (2021) Dynamic object aware lidar slam based on automatic generation of training data. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. IEEE, pp. 11641–11647.

Pfreundschuh P, Oleynikova H, Cadena C, Siegwart R and Andersson O (2024) Coin-lio: Complementary intensity-augmented lidar inertial odometry. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. IEEE, pp. 1730–1737.

Pomerleau F, Krüsi P, Colas F, Furgale P and Siegwart R (2014) Long-term 3d map maintenance in dynamic environments. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. IEEE, pp. 3712–3719.

Ramezani M, Wang Y, Camurri M, Wisth D, Mattamala M and Fallon M (2020) The newer college dataset: Handheld lidar, inertial and vision with ground truth. In: *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. pp. 4353–4360. DOI:10.1109/IROS45743.2020.9340849.

Rasmussen CE and Williams CKI (2006) *Gaussian Processes for Machine Learning*. The MIT Press.

Reijgwart V, Millane A, Oleynikova H, Siegwart R, Cadena C and Nieto J (2020) Voxgraph: Globally consistent, volumetric mapping using signed distance function submaps. *IEEE Robotics and Automation Letters (RA-L)* 5(1): 227–234. DOI: 10.1109/LRA.2019.2953859.

Ruan J, Li B, Wang Y and Sun Y (2023) Slamesh: Real-time lidar simultaneous localization and meshing. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. pp. 3546–3552. DOI:10.1109/ICRA48891.2023.10161425.

Sandström E, Li Y, Van Gool L and R Oswald M (2023) Point-slam: Dense neural point cloud-based slam. In: *Proc. of the IEEE Intl. Conf. on Computer Vision (ICCV)*.

Schmid L, Andersson O, Sulser A, Pfreundschuh P and Siegwart R (2023) Dynablox: Real-time detection of diverse dynamic objects in complex environments. *IEEE Robotics and Automation Letters (RA-L)* 8(10): 6259–6266. DOI:10.1109/LRA.2023.3305239.

Schmid L, Montiel JMM, Huang S, Cremers D, Neira J and Civera J (2026) Dynamic and deformable SLAM. In: Carlone L, Kim A, Barfoot T, Cremers D and Dellaert F (eds.) *SLAM Handbook. From Localization and Mapping to Spatial Intelligence*. Cambridge University Press.

Schmid L, Reijgwart V, Ott L, Nieto J, Siegwart R and Cadena C (2021) A unified approach for autonomous volumetric exploration of large scale environments under severe odometry drift. *IEEE Robotics and Automation Letters (RA-L)* 6(3): 4504–4511. DOI:10.1109/LRA.2021.3068954.

Segal A, Haehnel D and Thrun S (2009) Generalized-ICP. *Robotics: Science and Systems* 5: 168–176. DOI:10.1.1.149.3870.

Shan T and Englot B (2018) LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain. *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)* : 4758–4765.

Shan T, Englot B, Meyers D, Wang W, Ratti C and Rus D (2020) Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping. In: *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. pp. 5135–5142. DOI:10.1109/IROS45743.2020.9341176.

Talbot W, Nubert J, Tuna T, Cadena C, Dümbgen F, Tordesillas J, Barfoot TD and Hutter M (2025) Continuous-time state estimation methods in robotics: A survey. *IEEE Trans. on Robotics (TRO)* 41: 4975–4999. DOI:10.1109/TRO.2025.3593079.

Tang TY, Yoon DJ and Barfoot TD (2019) A white-noise-on-jerk motion prior for continuous-time trajectory estimation on se(3). *IEEE Robotics and Automation Letters (RA-L)* 4(2): 594–601.

Tereshkov VM (2015) A Simple Observer for Gyro and Accelerometer Biases in Land Navigation Systems. *Journal of Navigation* 68(04): 635–645. DOI:10.1017/s0373463315000016.

Umeyama S (1991) Least-squares estimation of transformation parameters between two point patterns. *IEEE Trans. on Pattern Analysis and Machine Intelligence (TPAMI)* 13(4): 376–380. DOI:10.1109/34.88573.

Vizzo I, Guadagnino T, Behley J and Stachniss C (2022) Vdbfusion: Flexible and efficient tsdf integration of range sensor data. *Sensors* 22(3): 1296.

Vizzo I, Guadagnino T, Mersch B, Wiesmann L, Behley J and Stachniss C (2023) KISS-ICP: In Defense of Point-to-Point ICP – Simple, Accurate, and Robust Registration If Done the Right Way. *IEEE Robotics and Automation Letters (RA-L)* 8(2): 1029–1036. DOI:10.1109/LRA.2023.3236571.

Wiesmann L, Guadagnino T, Vizzo I, Zimmerman N, Pan Y, Kuang H, Behley J and Stachniss C (2023) LocNDF: Neural Distance Field Mapping for Robot Localization. *IEEE Robotics and Automation Letters (RA-L)* 8(8): 4999–5006. DOI:10.1109/LRA.2023.3291274.

Williams O and Fitzgibbon AW (2006) Gaussian process implicit surfaces.

Wu L, Le Gentil C and Vidal-Calleja T (2025) Vdb-gpdf: Online gaussian process distance field with vdb structure. *IEEE Robotics and Automation Letters (RA-L)* 10(1): 374–381. DOI:10.1109/LRA.2024.3505814.

Wu L, Lee KMB, Le Gentil C and Vidal-Calleja T (2023) Log-gpis-mop: A unified representation for mapping, odometry, and planning. *IEEE Trans. on Robotics (TRO)* 39(5): 4078–4094. DOI:10.1109/TRO.2023.3296982.

Wu L, Lee KMB, Liu L and Vidal-Calleja T (2021) Faithful euclidean distance field from log-gaussian process implicit surfaces. *IEEE Robotics and Automation Letters (RA-L)* 6(2): 2461–2468. DOI:10.1109/LRA.2021.3061356.

Wu R, Pang C, Wu X and Fang Z (2024) Observation time difference: an online dynamic objects removal method for ground vehicles. *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)* : 17997–18003.

Xu W, Cai Y, He D, Lin J and Zhang F (2022) Fast-lio2: Fast direct lidar-inertial odometry. *IEEE Trans. on Robotics (TRO)* 38(4): 2053–2073. DOI:10.1109/TRO.2022.3141876.

Yang Y, Pious B, Babu W, Chen C, Huang G and Ren L (2020) Analytic Combined IMU Integration (ACI2) For Visual Inertial Navigation. *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)* .

Ye H, Chen Y and Liu M (2019) Tightly coupled 3D Lidar inertial odometry and mapping. *Proc. of the IEEE Intl. Conf. on*

Robotics & Automation (ICRA) 2019-May: 3144–3150. DOI: 10.1109/ICRA.2019.8793511.

Yoon D, Tang T and Barfoot T (2019) Mapless online detection of dynamic objects in 3d lidar. In: *Proc. of the Conf. on Robots and Vision*. IEEE, pp. 113–120.

Zäschke T, Zimmerli C and Norrie MC (2014) The ph-tree: a space-efficient storage structure and multi-dimensional index. In: *Proc. of the ACM SIGMOD International Conference on Management of Data*, SIGMOD '14. New York, NY, USA: Association for Computing Machinery. ISBN 9781450323765, p. 397–408. DOI:10.1145/2588555.2588564.

Zhang J and Singh S (2014) LOAM : Lidar odometry and mapping in real-time. *Robotics: Science and Systems* : 7–15DOI:10.1007/s10514-016-9548-2.

Zhang Q, Khoche A, Yang Y, Ling L, Mansouri SS, Andersson O and Jensfelt P (2025) Himo: High-speed objects motion compensation in point clouds. *IEEE Trans. on Robotics (TRO)* 41: 5896–5911. DOI:10.1109/TRO.2025.3619042.

Zhu D, Wang C, Wang W, Garg R, Scherer S and Meng MQH (2021) Vdb-edt: An efficient euclidean distance transform algorithm based on vdb data structure. *arXiv preprint* .

Zhu J, Li H, Wang Z, Wang S and Zhang T (2024) i-octree: A fast, lightweight, and dynamic octree for proximity search. In: *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. pp. 12290–12296. DOI:10.1109/ICRA57147.2024.10611019.

A Appendix

A.1 Distance field uncertainty proxy

It is important to note that the distance field from previous work (Le Gentil et al. 2024b) is not a GP as it is obtained via non-linear operations over a GP latent space. Accordingly, the naive GP variance inference (Rasmussen and Williams 2006) does not provide satisfying uncertainty information about the distance values $d(\mathbf{x})$ away from the surface. Here, after reviewing the proxy mechanisms previously introduced (Le Gentil et al. 2024b), we present a novel uncertainty proxy for the distance field.

To account for the uncertainty of the input point cloud, previous work introduced a method for optimizing the observation noise σ based on a typical data sample (Le Gentil et al. 2024b). While this is a principled way to address the issue of noisy inputs, it presents limitations. One is the uniqueness of the noise parameter σ, which hinders accuracy when considering non-stationary noise (both in time and space). In realistic lidar-based odometry scenarios, some surfaces will appear 'thicker' than others in the incoming point cloud due to various factors such as the noise in the lidar pose estimate, the angle of incidence of the lidar beams, etc. Regarding the uncertainty of the distance estimates, Le Gentil et al. (2024b) provides an elegant uncertainty proxy based on the degree of similarity between the field's gradient and the expected gradient. However, this mostly addresses the issue of over-interpolation in tight spaces (corridor-like pattern with a width in the order of magnitude of the kernel's lengthscale). We illustrate the aforementioned drawbacks in Fig. 16(c). Note that this proxy is not really relevant to the proposed efficient distance field, as the GPs are computed locally, considering a small neighbourhood, thus rarely presenting corridor-like patterns.

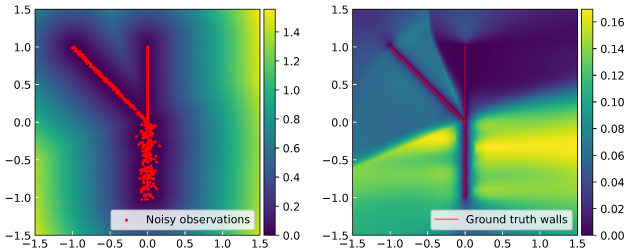
Conceptually, the use of GP regression for point-cloud-based distance field estimation does not fit the original purpose of standard GP regression: we extrapolate the field far from the data observations, whereas GP regression is an interpolation technique. In other words, we are seeking information and its variance in areas of space far from where the information is actually observed.[‖] The proposed distance uncertainty proxy 'fetches' the information where it is relevant for a given distance query by first selecting the closest surface point to the query location (denoted $\mathbf{s}(\mathbf{x})$ in the following). The location of $\mathbf{s}(\mathbf{x})$ can easily be deduced from the inferred distance and gradient $\mathbf{s}(\mathbf{x}) = \mathbf{x} - d(\mathbf{x})\nabla d(\mathbf{x})$. The following step consists of comparing the local shape of the latent field o around $\mathbf{s}(\mathbf{x})$ with that of a noiseless surface observation. We propose to use a local integration of the latent field to summarise the local shape of the latent field $\nu(\mathbf{s}(\mathbf{x})) = \iiint_{\mathbf{u}\in\mathcal{S}_{\mathbf{s}(\mathbf{x})}} o(\mathbf{u})d\mathbf{u}$, with $\mathcal{S}_{\mathbf{s}(\mathbf{x})}$ a small sphere around $\mathbf{s}(\mathbf{x})$. The integrals can be obtained with linear operators on the GP inference. Eventually, the proxy is defined as $\phi(\mathbf{x}) = |\nu(\mathbf{s}(\mathbf{x})) - \nu_{\text{calib}}|$, where ν_{calib} is computed with noiseless simulated data of a flat wall. It is equal to zero when the distance prediction is trustworthy and grows when the local shape differs from the ideal noiseless wall data.

The proposed proxy can be computed with the standard global GP formulation (Le Gentil et al. 2024b), but can also be made efficient with the sparse structure used in this work. First, the computation of $\mathbf{s}(\mathbf{x})$ can be omitted and approximated with the results of the spatial index query that already occurred to find the closest map voxel. Then, as the GP is computed locally, the integral over the sphere around $\mathbf{s}(\mathbf{x})$ can be replaced by the integral over the full $\mathbb{R}^3$ space. As the integral over $\mathbb{R}^3$ of the kernel function is a constant c, $\nu(\mathbf{s}(\mathbf{x})) \approx c\sum_i \alpha_i$, with $\boldsymbol{\alpha} = \left(\mathbf{K}_{\mathbf{X},\mathbf{X}} + \sigma\mathbf{I}\right)^{-1}\mathbf{1}$ (already computed in (16)). In the end, the additional computation cost of our novel uncertainty proxy is negligible. An example of the proposed proxy is given in Fig. 16 (d). Note that in its current state, the proxy does not directly provide the standard deviation of the inferred distance. For this reason, we decided not to account for it in 2Fast-2Lamaa's distance-field-based scan-to-map registration. Investigating the physical meaning of ϕ is outside the scope of this paper and is part of future work.

A.2 Voxel weighting analysis

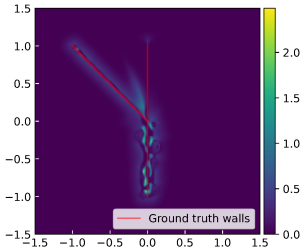
An appealing aspect of GP regression is its ability to handle input noise elegantly. With the simplifying assumption that lidar points are subject to independent and identically distributed (i.i.d.) spherical noise, considering that the 'surface measurements' in (16) are also i.i.d. is a sensible approach. The underlying intuition is that every lidar point is as informative as any other lidar point. However, when voxelizing geometric data for computational efficiency, this assumption of equal information for every input point

[‖] Please note that the GP-inferred variance is still relevant close to the surface and can directly be leveraged in applications such as surface reconstruction.

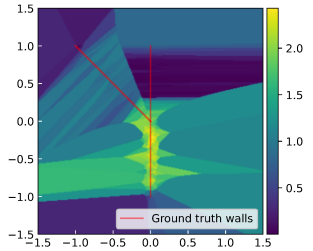


(a) Inferred distance [m]

(b) Distance field error [m]

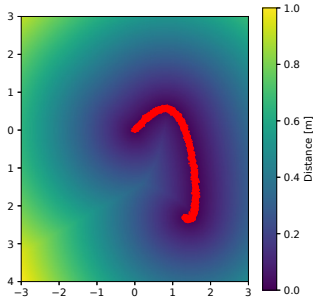


(c) Existing uncertainty proxy (Le Gentil et al. 2024b)

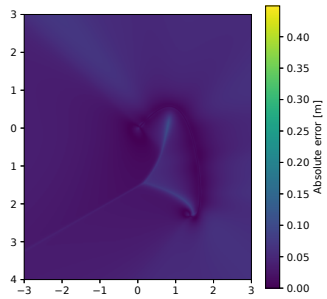


(d) Novel uncertainty proxy

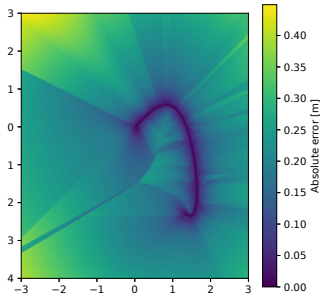
Figure 16. Visualization of the proposed uncertainty proxy when considering non-stationary observation noise. (a) shows the estimated distance field with the observation parameters tuned for the least noisy of the wall observations. (b) shows the error compared with the ground-truth. (c) is the uncertainty proxy introduced in previous work (Le Gentil et al. 2024b). (d) is the proposed uncertainty proxy, which patterns match the area of low and high error in (b).



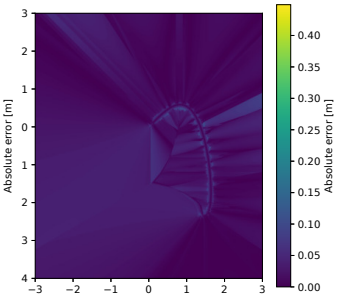
(a) Ground-truth distance field and noisy observations



(b) Absolute inference error dense GP (Le Gentil et al. 2024b)



(c) Absolute inference error ours unweighted



(d) Absolute inference error ours weighted

Figure 17. Visualization of the distance field inference error with *dense GP*, and our proposed efficient distance function with and without the proposed weighting mechanism.

Table 13. Average RMSE error of the proposed efficient distance field and various baselines.

Variant	Average RMSE [m]	Std. [m]
Voxel NN	0.072	0.006
Dense GP (Le Gentil et al. 2024b)	0.047	0.007
Ours unweighted	0.252	0.019
Ours weighted	**0.029**	0.006

is not valid. Each voxel's centroid provided a different amount of information depending on the number of points that occurred in it. Thus, we presented in Section 6.2 a weighting mechanism that adapts the GP's measurement noise model to account for discrepancies in the individual voxel's information. In this appendix, we provide a brief comparison of accuracy with and without the proposed weighting scheme. We also compare the results of the proposed distance field against a simple nearest-neighbour search and the dense GP formulation from previous work (Le Gentil et al. 2024b).

Our 2D simulated setup emulates lidar points collected along a randomly-shaped wall. The wall is defined as a sum of sine functions. A 3 cm spherical Gaussian noise is applied to each of the 10k points of the scene. Fig. 17(a) shows one of the randomly generated scenes. The distance field's ground-truth is computed based on a nearest-neighbour search with high-density noiseless wall measurements. Table 13 shows the average distance RMSE over 100 runs. Note that the parameters of the *Dense GP* version are tuned specifically for the simulated density and noise distribution. The *unweighted* (considering each voxel equally informative with an i.i.d. observation noise) and *weighted* versions share the same generic parameters (cell size, lengthscale, etc) to highlight the benefits of the proposed weighting mechanism. A smaller error could be achieved with the *unweighted* variant by specifically tuning the parameter to the point density and noise distribution of this setup, but losing genericity. As illustrated in the error plot in Fig. 17, the *unweighted* approach creates a virtually 'thicker' surface, offsetting the distance values inferred away from the wall. Interestingly, as the proposed efficient distance field method considers small local neighbourhoods to compute the GPs, the method is somewhat unaffected by the phenomenon of over-interpolation in narrow spaces that degrades the performance of the *dense* formulation. Accordingly, our method performs the best, at least on this simulated setup. As expected, the proposed approach performs significantly better than a simple nearest-neighbour search (*Voxel NN*), which correlates with the final odometry findings in our ablation study (Section 9.6).

A.3 Spatial index performance

In this appendix, we benchmark spatial indexes that fulfill 2Fast-2Lamaa's requirements, which are the ability to add or remove points without rebuilding the index from scratch, allowing k-nearest-neighbour and radius searches, and scalability with over 10 million points in the index. We use three data structures with publicly available C++ implementations: ikd-Trees (Cai et al. 2021), PH-Trees (Zäschke et al. 2014), and i-Octrees (Zhu et al. 2024). Our toy example emulates 2Fast-2Lamaa's use case in driving scenarios with an incrementally built map of 10 million points created along a 5 km trajectory. We simulate the lidar collection with 'scans' of 20k points at random

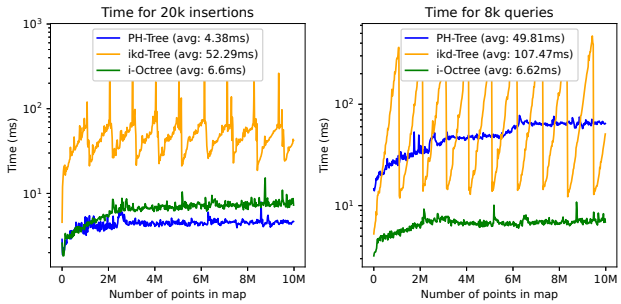


Figure 18. Timing comparison, using a log-scale, between ikd-Tree, PH-Tree, and i-Octree for point insertion and nearest neighbour search in a growing map (using scans made of random points in a virtual $5\,\mathrm{km}$ straight line trajectory).

locations within the lidar range ($100\,\mathrm{m}$). Every time a scan is inserted in the spatial index, 8k nearest-neighbour queries are performed to imitate the registration process. Dividing the goal of 10 million points in the final map by the size of the scans, we obtain 500 steps of $10\,\mathrm{m}$ each. The trajectory is defined as a straight line parameterized by $x = y$ and $z = 0$. Fig. 18 shows the timing obtained with the three methods using a single core of our Intel i7-1370p CPU.** The results show an advantage for the PH-Tree for insertion, but the i-Octree clearly outperforms the other two methods for queries. Note that using multi-threading on the queries can greatly reduce the computation time of i-Octree queries down to $1.51\,\mathrm{ms}$ (with 8 cores). The jagged nature of the ikd-Tree reflects its need for regular balancing.

A.4 Entire Boreas-RT dataset results

This appendix provides the odometry and localization results obtained with 2Fast-2Lamaa on the entirety of the Boreas-RT dataset (Lisus et al. 2026).

A.4.1 Odometry Tables 14, 15, and 16 corresponds to the *Structured*, *Rural*, and *Regional* sequence types, respectively. In total, nearly $650\,\mathrm{km}$ of data is used to generate these results. 2Fast-2Lamaa robustly provides high-accuracy estimates, consistently outperforming LTR (Burnett et al. 2022).

A.4.2 Localization The localization results are provided in Table 17. Note that the *Forest* and *Urban* routes are omitted as the ground-truth trajectories are not sufficiently consistent between repeated sequences to enable centimetre-level localization evaluation. As explained by Lisus et al. (2026), a lack of unobstructed view of the sky, due to trees or high buildings, and the distance to the RTK base station lead to larger errors in the sequence-to-sequence ground-truth accuracy (greater than $0.4\,\mathrm{m}$).

2Fast-2Lamaa and LTR (see Lisus et al. (2026) for results) provide similar levels of accuracy and robustness in structured environments such as *Suburbs*, *Industrial*, and *Regional*. However, in more challenging environments, LTR reports larger errors and even fails on some sequences. The variability of the vertical accuracy on the *Suburbs* and *Farm* is explained by significant changes in the environment between the time of mapping and the time of localization. The former is due to large snowbanks that were created by snowstorms. The latter is due to the vegetation in the field that grew during the month separating the two data-collection days. Overall, 2Fast-2Lamaa provides an average RMSE lateral error under $0.05\,\mathrm{m}$.

Table 14. Per-sequence odometry results on the *Structured* sequences of the Boreas-RT dataset.

Sequence type	ID	LTR w/ gyro	2Fast-2Lamaa
suburbs	2024-12-03-12-54	0.30/0.10	0.24/0.08
	2024-12-05-14-25	0.29/0.09	0.21/0.08
	2025-01-08-10-59	0.30/0.09	0.23/0.08
	2025-01-08-11-22	0.28/0.09	0.21/0.07
	2025-01-08-12-28	0.26/0.08	0.21/0.08
	2025-02-15-16-58	0.27/0.09	0.20/0.07
	2025-02-15-17-19	0.27/0.09	0.22/0.08
	2025-02-21-14-51	0.34/0.10	0.24/0.08
	2025-02-22-11-32	0.29/0.09	0.25/0.09
	2025-02-22-12-26	0.30/0.10	0.22/0.08
	Average	0.29/0.09	**0.22/0.08**
industrial	2024-12-05-14-12	0.23/0.08	0.21/0.08
	2024-12-23-16-27	0.27/0.10	0.21/0.08
	2024-12-23-16-44	0.24/0.09	0.20/0.08
	2024-12-23-17-01	0.26/0.09	0.19/0.07
	2024-12-23-17-18	0.24/0.09	0.19/0.07
	Average	0.25/0.09	**0.20/0.07**
urban	2025-08-06-06-33	0.41/0.16	0.35/0.11
	2025-08-06-07-05	0.46/0.17	0.40/0.11
	2025-08-06-07-41	0.43/0.17	0.37/0.11
	2025-08-06-08-35	0.55/0.22	0.53/0.11
	2025-08-06-10-48	0.48/0.21	0.31/0.10
	2025-08-06-11-32	0.57/0.23	0.39/0.10
	2025-08-06-12-20	0.51/0.24	0.29/0.09
	Average	0.49/0.20	**0.38/0.10**

KITTI odometry metric reported as *XX / YY* with *XX* [%] and *YY* [$^\circ/100\,\mathrm{m}$] the translation and orientation errors, respectively.

**Code available here: https://github.com/clegenti/spatial_index_benchmark

Table 15. Per-sequence odometry results on the *Rural* sequences of the Boreas-RT dataset.

Sequence type	ID	LTR w/ gyro	2Fast-2Lamaa
forest	2025-07-18-10-33	0.66/0.14	0.46/0.18
	2025-07-18-11-00	0.71/0.10	0.46/0.18
	2025-07-18-11-25	0.64/0.10	0.48/0.18
	2025-07-18-11-53	0.78/0.10	0.47/0.19
	Average	0.70/**0.11**	**0.47**/0.18
farm	2025-07-18-14-55	0.45/0.11	0.35/0.10
	2025-07-18-15-12	0.44/0.10	0.37/0.10
	2025-07-18-15-30	0.42/0.11	0.38/0.10
	2025-07-18-15-48	0.60/0.11	0.39/0.11
	2025-07-18-16-05	0.49/0.10	0.40/0.11
	2025-08-13-09-01	0.40/0.13	0.33/0.10
	2025-08-13-09-21	0.40/0.13	0.34/0.09
	2025-08-13-09-46	0.42/0.13	0.34/0.09
	2025-08-13-10-12	0.42/0.13	0.35/0.10
	2025-08-13-10-36	0.40/0.12	0.34/0.10
	Average	0.44/0.12	**0.36/0.10**

KITTI odometry metric reported as *XX / YY* with *XX* [%] and *YY* [$^\circ/100\,\mathrm{m}$] the translation and orientation errors, respectively.

Table 16. Per-sequence odometry results on the *Highway* sequences of the Boreas-RT dataset.

Sequence type	ID	LTR w/ gyro	2Fast-2Lamaa
regional	2024-12-03-13-13	0.34/0.09	0.18/0.07
	2024-12-03-13-34	0.34/0.09	0.20/0.08
	2024-12-10-12-07	0.34/0.09	0.20/0.08
	2024-12-10-12-24	0.35/0.10	0.21/0.08
	2024-12-10-12-38	0.32/0.08	0.18/0.08
	2024-12-10-12-56	0.38/0.10	0.21/0.08
	Average	0.35/0.09	**0.20/0.08**
tunnel	2024-12-04-14-28	2.26/0.12	0.27/0.08
	2024-12-04-14-34	X	0.60/0.17
	2024-12-04-14-38	2.82/0.11	0.29/0.10
	2024-12-04-14-44	1.69/0.08	0.40/0.14
	2024-12-04-14-50	X	0.30/0.10
	2024-12-04-14-59	2.05/0.09	0.37/0.11
	2024-12-04-15-04	2.36/0.11	0.26/0.09
	2024-12-04-15-10	1.02/0.09	0.33/0.11
	2024-12-04-15-19	2.87/0.11	0.28/0.09
	2024-12-04-15-24	1.41/0.09	0.39/0.14
	Average	2.06/**0.10**	**0.35**/0.11
skyway	2024-12-04-11-45	1.62/0.08	0.30/0.10
	2024-12-04-11-56	0.47/0.07	0.32/0.10
	2024-12-04-12-08	0.63/0.08	0.26/0.09
	2024-12-04-12-19	0.51/0.08	0.34/0.10
	2024-12-04-12-34	0.60/0.08	0.27/0.09
	Average	0.77/**0.08**	**0.30**/0.09
freeway	2025-07-18-16-24	0.53/0.13	0.26/0.10
	2025-08-13-07-54	0.42/0.11	0.20/0.07
	2025-08-13-11-52	0.44/0.13	0.26/0.10
	Average	0.46/0.12	**0.24/0.09**

KITTI odometry metric reported as *XX / YY* with *XX* [%] and *YY* [$^\circ/100\,\mathrm{m}$] the translation and orientation errors, respectively.

Table 17. Per-sequence RMSE localization results for 2Fast-2Lamaa on the Boreas-RT dataset.

Seq. type & map seq.	Seq. ID	Long. [m]	Lat. [m]	Vert. [m]	Roll [°]	Pitch [°]	Yaw [°]
Suburbs	2024-12-05-14-25	0.031	0.023	0.041	0.039	0.032	0.030
2024-12-03-12-54	2025-01-08-10-59	0.027	0.024	0.040	0.042	0.029	0.029
	2025-01-08-11-22	0.024	0.021	0.025	0.038	0.028	0.025
	2025-01-08-12-28	0.026	0.025	0.057	0.045	0.031	0.027
	2025-02-15-16-58	0.025	0.027	0.230	0.078	0.040	0.030
	2025-02-15-17-19	0.025	0.031	0.234	0.078	0.044	0.032
	2025-02-21-14-51	0.030	0.028	0.280	0.082	0.052	0.030
	2025-02-22-11-32	0.030	0.027	0.278	0.084	0.047	0.029
	2025-02-22-12-26	0.032	0.026	0.278	0.083	0.067	0.028
	Average	0.028	0.026	0.163	0.063	0.041	0.029
Regional north	2024-12-10-12-07	0.032	0.028	0.031	0.037	0.026	0.030
2024-12-03-13-13	2024-12-10-12-38	0.035	0.043	0.041	0.038	0.033	0.030
	Average	0.034	0.035	0.036	0.038	0.030	0.030
Regional south	2024-12-10-12-24	0.031	0.026	0.046	0.035	0.026	0.025
2024-12-03-13-34	2024-12-10-12-56	0.031	0.029	0.034	0.041	0.028	0.029
	Average	0.031	0.028	0.040	0.038	0.027	0.027
Tunnel east	2024-12-04-14-38	0.066	0.025	0.060	0.077	0.025	0.031
2024-12-04-14-28	2024-12-04-14-50	0.119	0.028	0.032	0.089	0.024	0.029
	2024-12-04-15-04	0.112	0.030	0.046	0.087	0.026	0.032
	2024-12-04-15-19	0.095	0.025	0.046	0.080	0.024	0.029
	Average	0.098	0.027	0.046	0.083	0.024	0.030
Tunnel west	2024-12-04-14-34	0.136	0.024	0.043	0.150	0.028	0.029
2024-12-04-14-44	2024-12-04-14-59	0.230	0.028	0.022	0.081	0.020	0.028
	2024-12-04-15-10	0.079	0.043	0.061	0.084	0.024	0.038
	2024-12-04-15-24	0.176	0.053	0.039	0.113	0.041	0.033
	Average	0.155	0.037	0.041	0.107	0.028	0.032
Skyway	2024-12-04-11-56	0.126	0.047	0.050	0.186	0.040	0.083
2024-12-04-11-45	2024-12-04-12-08	0.062	0.035	0.052	0.058	0.038	0.031
	2024-12-04-12-19	0.074	0.038	0.047	0.067	0.039	0.035
	2024-12-04-12-34	0.067	0.035	0.044	0.062	0.038	0.040
	Average	0.082	0.038	0.048	0.093	0.039	0.048
Industrial	2024-12-23-16-27	0.024	0.023	0.027	0.063	0.040	0.034
2024-12-05-14-12	2024-12-23-16-44	0.023	0.022	0.038	0.066	0.039	0.030
	2024-12-23-17-01	0.023	0.022	0.038	0.063	0.040	0.038
	2024-12-23-17-18	0.021	0.021	0.036	0.065	0.042	0.034
	Average	0.023	0.022	0.035	0.064	0.040	0.034
Farm	2025-07-18-15-12	0.046	0.040	0.058	0.052	0.035	0.033
2025-07-18-14-55	2025-07-18-15-30	0.045	0.040	0.063	0.054	0.036	0.030
	2025-07-18-15-48	0.048	0.044	0.071	0.052	0.039	0.038
	2025-07-18-16-05	0.049	0.041	0.068	0.053	0.037	0.033
	2025-08-13-09-01	0.085	0.068	0.169	0.175	0.060	0.069
	2025-08-13-09-21	0.079	0.073	0.165	0.167	0.057	0.042
	2025-08-13-09-46	0.085	0.067	0.176	0.173	0.058	0.047
	2025-08-13-10-12	0.079	0.060	0.187	0.174	0.057	0.049
	2025-08-13-10-36	0.071	0.059	0.185	0.174	0.056	0.062
	Average	0.065	0.055	0.127	0.119	0.048	0.045
Freeway	2025-08-13-11-52	0.052	0.038	0.082	0.067	0.033	0.037
2025-07-18-16-24	**Average**	0.052	0.038	0.082	0.067	0.033	0.037